



HASHDIT

HashDit

Smart Contract Security

ListaDao V3 LP Collateral

Security Audit Report

ListaDao

29 Jun 2026 – 14 Jul 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	29 Jun 2026 – 14 Jul 2026
Overall Risk [Optional]	High
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/151
Commit	110544000c86e8bdf36971e7b7cdb3ec93a672f7

Smart Contract List

No.	Contract Name	Verdict	Details
1	V3Provider.sol	See Findings	See Findings
2	V3DexAdapter.sol	See Findings	See Findings
3	V3ProviderOracle.sol	See Findings	See Findings
4	SlisBNBV3Provider.sol	See Findings	See Findings
5	SlisBNBV3DexAdapter.sol	See Findings	See Findings
6	SlisBNBV3ProviderOracle.sol	See Findings	See Findings
7	V3Liquidator.sol	See Findings	See Findings
8	WstETHV3Provider.sol	See Findings	See Findings
9	WstETHV3DexAdapter.sol	See Findings	See Findings
10	WbETHV3Provider.sol	See Findings	See Findings
11	WbETHV3DexAdapter.sol	See Findings	See Findings
12	V3PositionLib.sol	See Findings	See Findings
13	SwapInventoryLib.sol	See Findings	See Findings
14	SmartProvider.sol	See Findings	See Findings

Findings

V3Provider

[H01] Zero slippage compounding of accumulated fees in the single V3 position can be sandwiched to extract value from all share holder

Contract V3Provider.sol

Severity **High**

Description

The V3DexAdapter holds a single Uniswap or PancakeSwap V3 position that continuously accrues swap fees. Those fees are only compounded back into the position by collectAndCompound via the internal _collectAndCompound, which runs solely as a side effect of deposit, withdraw, redeemShares, and rebalance invoked within the V3Provider contract. There is no standalone compounding function.

Description

JavaScript

```
function _collectAndCompound() internal {
    if (tokenId == 0) return;

    (uint256 fees0, uint256 fees1) =
V3PositionLib.collectAll(POSITION_MANAGER, tokenId);

    uint256 toCompound0 = fees0 + idleToken0;
    uint256 toCompound1 = fees1 + idleToken1;
    if (toCompound0 == 0 && toCompound1 == 0) return;

    (uint128 liquidityAdded, uint256 used0, uint256 used1) =
V3PositionLib.increaseLiquidity(
    POSITION_MANAGER,
    TOKEN0,
    TOKEN1,
    tokenId,
    toCompound0,
    toCompound1,
    0,
    0
);

    idleToken0 = toCompound0 - used0;
    idleToken1 = toCompound1 - used1;

    emit Compounded(toCompound0, toCompound1, liquidityAdded);
}
```

For a lending collateral vault deposits and withdrawals can be infrequent if users decide to deposit and hold, and rebalances are expected to occur only during roughly 1% exchange rate drift, which could take a substantial period of time to occur depending on pool prices. As a result, fees could accumulate uncompounded in the single position for long periods, and the entire accumulated amount is later added back in one operation.

Impact

`collectAndCompound` performs that add through `increaseLiquidity` with zero slippage protection, passing `amount0Min = 0` and `amount1Min = 0`.

Because it executes at the current pool spot price and can be triggered by anyone through a minimal deposit or withdrawal, where the user sets its own loose slippage so it does not revert (via the `V3Provider` entry points of `deposit`, `withdraw`, `redeemShares`, and `rebalance`), an attacker can move the pool price with a flashloan, trigger the compound so that the accumulated fees plus idle inventory are added at the manipulated price, then revert the price and capture the impermanent loss on the freshly added liquidity. The value at risk scales with the accumulated fees plus idle, which grows with the time since the last compound. The uncompounded fees are also not reflected in the reported collateral value, since `positionAmountsAt` reads only the last checkpointed `tokensOwed`, so the accumulation is invisible until it is compounded.

For the `slisBNB` and `WBNB` pair the managed range is $\pm 1\%$, so the extractable amount per attack is limited to roughly 1% of the accumulated amount, which caps the loss for that specific pair.

However `V3Provider`, `V3DexAdapter`, and the oracle are designed as generic contracts intended to support any V3 LP collateral position. For the wider ranges those generic pairs use, for example the roughly 5% fallback range of 500 ticks around spot, the extractable fraction is several times larger, and the fair price for such pairs falls back to the pool TWAP, which is itself manipulable. Combined with a thin, easily moved pool and a flashloan, this allows material extraction of value from all share holders on every accumulated compound. The same zero slippage weakness also applies to the `rebalance re-mint`, which adds the entire position rather than only fees, further increasing exposure.

Consider the following:

Recommendation

1. Gate `collectAndCompound` and the `rebalance re-mint` on a spot versus fair price deviation check, reverting or skipping the compound when the pool spot deviates from `fairSqrtPriceX96` beyond a manager configured threshold. For the rate anchored pairs this is robust because a flashloan can move the pool spot but cannot move the exchange rate that defines the fair price.

2. Additionally, add a permissionless compounding function guarded by the same deviation check, so accrued fees are swept frequently and never accumulate into a single large amount that can be sandwiched. For the generic pairs, derive the fair price from the resilient oracle rather than the pool TWAP, and make the range width a configurable per pair parameter so the extractable fraction stays bounded.

Status

Fixed, now utilizes a fair V3DexAdapter_collectAndCompound that performs initial checks that the pool spot price is within the a admin set maxSpotDeviationBps, ensuring that fees are only allowed to be compounded if current spot price is within the allowed deviation from the fairSqrtPriceX96 via _spotWithinFair. This enforces that when liquidity addition of compounded fees/idle tokens is performed, it is performed at a pool spot price within an allowed safe deviation range, not at an attacker's manipulated pool price.

[H02] First depositor inflation attack when the accounting asset has low decimals relative to its price**Contract** V3Provider.sol**Severity** **High****Description**

For deposits after the first, share amount is computed with the raw proportional formula $\text{shares} = \text{addedValue} * \text{supplyBefore} / \text{totalValueBefore}$, and for the first deposit with $\text{shares} = \text{addedValue} * 10^{**} \text{accountingAssetDecimals} / \text{assetPrice}$, where addedValue is an integer valued in 8 decimal USD and assetPrice is the accounting asset price in 8 decimal USD. The custom deposit path does not use the ERC4626 convertToShares, so the inherited virtual shares and decimal offset protection does not apply, and there is no minSharesOut guard. The ZeroShares check only reverts a mint of exactly 0 shares.

Description

JavaScript

```
(uint256 added0, uint256 added1) =
IV3DexAdapter(ADAPTER).amountsForLiquidity(liquidityAdded,
fairSqrtPriceX96);
uint256 addedValue = _amountsValueUsd(added0, added1);
if (supplyBefore == 0) {
    uint256 assetPrice = IOracle(resilientOracle).peek(asset()); // 8
    decimals
    if (assetPrice > 0) shares = (addedValue * (10 **
uint256(accountingAssetDecimals))) / assetPrice;
} else {
    shares = (addedValue * supplyBefore) / totalValueBefore;
}
if (shares == 0) revert ZeroShares();
```

The only thing preventing a first depositor inflation attack is the size of the smallest mintable supply, which is governed entirely by the factor $F = 10^{**} \text{accountingAssetDecimals} / \text{assetPrice}$. Since the smallest non zero addedValue is 1, the minimum first deposit supply equals $\max(1, \text{floor}(F))$. When the accounting asset has high decimals relative to its price this factor is large and acts as a big virtual share buffer, but when the accounting asset has low decimals relative to its price the factor collapses toward 1 and the attacker can seed a supply of a single share.

For example, the slisBNB and WBNB pair the accounting asset is WBNB, 18 decimals and roughly 600 USD, so $F = 1e18 / 6e10 \approx 1.67e7$. The minimum supply is about 16.7 million shares, the attack return is on the order of $1 / 1.67e7$, and the pair is not exploitable.

However, V3Provider, V3DexAdapter, and the oracle are designed as generic contracts intended to support any V3 LP collateral position, and the accounting asset is a deployment choice. A 6 decimal USDC at 1 USD gives $F = 1e6 / 1e8 = 0.01$, an 8 decimal token at 1 USD gives $F = 1e8 / 1e8 = 1$, and an 8 decimal WBTC at roughly 60000 USD gives $F \approx 1.7e-5$. In every one of these the minimum supply is 1.

Impact

When the minimum supply is 1 the attack is fully profitable. The attacker first deposits pool token dust sufficient for addedValue = 1, minting exactly 1 share and owning the entire supply. The attacker then inflates totalValueBefore without minting any shares by calling increaseLiquidity directly on the adapter's position, which has no authorization check, adding a donation D. This donation is not lost, because the attacker's shares claim it pro rata and the attacker owns essentially all shares, so it is recovered on exit. A victim then deposits v_v and receives $\text{floor}(v_v * \text{supplyBefore} / \text{totalValueBefore})$ shares, rounded down, with no minSharesOut to protect them and with ZeroShares only turning sub one share victims into reverts. The attacker withdraws and captures the rounded off value.

Worked example with an 8 decimal, 1 USD accounting asset and a 10000 USD victim: the attacker seeds 1 share, front runs with a donation of $D = 5001$ USD, the victim's shares compute to $\text{floor}(1e12 * 1 / 5.001e11) = 1$, so total supply becomes 2 and total value becomes 15001 USD. The victim's single share is worth 7500 USD against a 10000 USD deposit, a loss of 2500 USD, while the attacker's single share is also worth 7500 USD against a 5001 USD donation, a profit of 2499 USD, roughly a 50% return on briefly locked capital. The capital D is sized to the victim's deposit rather than being

	astronomical, and is fully recovered plus profit. The attack is only feasible in the launch window while total supply is still small, and is not enforced against on chain.
	Consider not allowing inflation resistance to depend implicitly on the accounting asset's decimals and price.
Recommendation	Add explicit protection to the custom mint path, since inheriting ERC4626 does not protect a formula that bypasses convertToShares. We can either add fixed virtual shares and virtual value to both supplyBefore and totalValueBefore in the proportional formula, or mint a fixed quantity of dead shares to a burn address on the first deposit, so the minimum supply is always sufficiently large to prevent first depositor inflation attacks. If not, always ensure that ListaDao is always the first seed depositor of every V3Provider market.
Status	Fixed, now includes a minShares input for share slippage that allows a user to input an expected share to receive in the event of a first depositor mint. Additionally, ListaDao has indicated that they will be the first depositor of every V3 Collateral market, so this issue is effectively mitigated.

[M01] Missing share price slippage protection on deposit

Contract	V3Provider.sol
Severity	Medium
Description	Description The V3Provider vault values its concentrated liquidity position at the rate implied fairSqrtPriceX96, but every deposit actually moves tokens inside the underlying pool at the pool spot price. On deposit, the shares minted are computed from addedValue, the newly added liquidity revalued at fairSqrtPriceX96 through the oracle. The tokens the user actually parts with are the amounts required to mint that liquidity at spot. <pre> JavaScript (uint256 added0, uint256 added1) = IV3DexAdapter(ADAPTER).amountsForLiquidity(liquidityAdded, fairSqrtPriceX96); uint256 addedValue = _amountsValueUsd(added0, added1); if (supplyBefore == 0) { uint256 assetPrice = IOracle(resilientOracle).peek(asset()); // 8 decimals if (assetPrice > 0) shares = (addedValue * (10 ** uint256(accountingAssetDecimals))) / assetPrice; } else { shares = (addedValue * supplyBefore) / totalValueBefore; } if (shares == 0) revert ZeroShares(); </pre>

Impact

For a fixed liquidity L valued in the token1 numeraire at the fair rate R_f , the required token value $f(P) = \text{amount0}(L,P) * R_f + \text{amount1}(L,P)$ reaches its minimum exactly when the pool price P equals the fair price. This means $f(P_{\text{spot}}) \geq f(P_{\text{fair}})$ for any deviation, with the gap growing roughly with the square of the deviation. The depositor therefore pays $f(P_{\text{spot}})$ in tokens but is credited only $f(P_{\text{fair}})$ in share value, overpaying the difference. That excess leaks to arbitrageurs and to the continuing shareholders. The existing `amount0Min` and `amount1Min` parameters do not stop this, as they bound only the input token ratio on the spot price, not the number of shares received. There is no minimum share parameter, so the depositor has no way to require a floor on shares received for the value supplied.

Several conditions widen the difference between spot to and fair price gap and therefore possibly cause a loss:

1. The paired LST appreciates monotonically against the native asset, so the pool spot persistently lags the rising fair price. Every depositor mints into a stale priced pool and is shortchanged in a repeated, directional way, while every exiter can over extract. (Latent)
2. A validator slashing event lowers the LST exchange rate, which decreases the fair price as well as the retrieved liquidity and priced value of that total liquidity.
3. Each rebalance can leak a small absolute amount of value through swap slippage, changing the position value between the moment a user quotes and the moment the transaction executes.
4. Oracle price movement or a break of the rate consistency assumption between the two legs can misprice the shares minted or the value credited.
5. An attacker can push the positive range within a single transaction, forcing a depositor to the worst corner of $f(P)$ or letting an exciter drain the maximum from remaining derived bounds.

Recommendation

Add a `minShares` floor and a deadline to deposit, and revert when the shares minted fall below the caller supplied minimum or the deadline has passed. This is the only guard denominated in what the depositor actually receives and the only one anchored to their fair value expectation rather than to spot, so it directly closes the overpayment gap and also catches adverse changes in position value, total supply, exchange rate, or oracle price between quote and execution.

Status

Fixed, now includes a `minShares` input for share slippage that allows a user to input an expected share to receive in the event of the above conditions that can return an unexpected share amount.

[L01] Missing self and system address validation in withdrawShares() and supplyShares() can permanently lock shares and leak unclaimable rewards

Contract V3Provider.sol

Severity Low

Description

Every user-facing entrypoint in V3Provider validates its recipient parameter only against the zero address and never against the protocol's own system addresses (the provider itself, the DEX adapter, and Moolah). Because the ERC4626 share token restricts public transfers to Moolah only, and because none of these system contracts expose a way to move the assets back out, any assets routed to them are lost permanently. The following entry points are affected.

Description

1. First, withdrawShares() validates only that the receiver is not the zero address. Setting receiver to the provider, the adapter, or Moolah sends the share token to an address with no spending path, locking the shares while they still count toward totalSupply, so their backing liquidity stays stranded in the adapter.
2. Second, supplyShares() validates only that onBehalf is not the zero address. Supplying on behalf of another arbitrary user is an irrevocable gift, since the caller always funds the shares and the recipient only gains collateral, which cannot steal funds or force a liquidation but loses the caller's shares on mistaken input. Supplying on behalf of the provider itself creates a Moolah collateral position owned by the provider that can never be withdrawn, because releasing a position for onBehalf = provider requires msg.sender == provider or MOOLAH.isAuthorized(provider, msg.sender) == true, and the provider never calls itself and never authorizes any address. The collateral change also triggers _syncPosition(provider), which mints slisBNBx to the provider through slisBNBxMinter.rebalance(provider), and since the provider cannot transfer or claim slisBNBx, that reward is locked forever and dilutes the pro-rata Launchpool reward share of legitimate users.
3. Third, deposit() validates only that onBehalf is not the zero address. Setting onBehalf to the provider produces the same permanent lock of the resulting collateral position and the same unclaimable, dilutive slisBNBx leak as the provider case of supplyShares(), except that the loss here is the depositor's freshly supplied token0 and token1 rather than pre-existing shares.
4. Fourth, withdraw() and redeemShares() send the withdrawn underlying token0 and token1 through the adapter directly to receiver, validated only against the zero address. Setting the receiver to the provider is the most dangerous case, because the slisBNB leg is transferredNB leg is unwrapped and delivered to the provider 's receive() accepts since the sender is the adapter. Both legs land in

the provider with no revert, and the provider has no function that uses or releases its raw token or native balance the entire withdrawn amount stranded.

Impact

None of these behaviors allow theft or forced liquidation of third parties, and none affect the share price for other holders, because the net asset value and the oracle price are computed from the adapter's position rather than from the provider's token balances, and totalSupply decreases in step with any position reduction. Stranded balances are never counted toward net asset value, are never swept into later flows, since amounts are always explicit and never balance based, and cannot be extracted by anyone.

Recommendation

Consider adding a shared validation that rejects the sensitive system addresses on the recipient parameter of every entrypoint. Reject receiver being the provider, the adapter, or Moolah in withdrawShares(), withdraw(), and redeemShares().

Status

Acknowledged, no fix implemented. It is presumed that users will not perform market actions routing shares to the V3Provider/Moolah contract itself, since it will only result in self-sabotage.

[L02] V3Provider is incompatible with fee on transfer tokens

Contract V3Provider.sol

Severity Low

Description

The generic V3Provider assumes that the amount requested for a token transfer equals the amount actually received. This assumption fails for fee on transfer (FoT) tokens, and the impact varies across the contract's market actions.

Description

- In deposit(), the vault first pulls the token from the user with safeTransferFrom, so under a FoT token it receives `_amount0Desired - fee`. It then forwards the full `_amount0Desired` to the adapter with `safeTransfer(ADAPTER, _amount0Desired)`. Because the vault keeps no standing inventory, its balance equals the fee reduced amount, which is `< _amount0Desired`, so the forwarding transfer reverts and deposits with a FoT token are fully bricked.
- A second FoT hop follows the same flawed assumption, where `addLiquidity` passes `amount0Desired` directly to the `NonfungiblePositionManager`, which pulls from the adapter, and the adapter also only received the fee reduced amount. The design therefore assumes `received = requested` at two separate hops.

Impact

All three in scope subclasses (`slisBNB/WBNB`, `wstETH/WETH`, `wbETH/WETH`) pair a wrapped native token with a non FoT LST, so no current deployment triggers this issue.

	It remains a latent problem for any future generic pair, for example if USDT enables its transfer fee
	Note: There are impacts on withdrawals and redemption of shares but since deposits are not reachable for such tokens, that impact is not highlighted.
Recommendation	Consider never supporting FOT tokens for the V3Provider, if not, we need to cache balances appropriately if allowed.
Status	Acknowledged, it is presumed that tokens with such characteristics will never be utilized when performing V3Provider market creations.

[L03] deposit refunds unused wrapped-native as native coin; non-payable receivers cause revert

Contract	V3Provider.sol
Severity	Low
Description	Description When a depositor funds the wrapped-native leg with msg.value, V3Provider.deposit() wraps the native coin into the wrapped-native ERC-20, forwards it to the adapter for addLiquidity, mints shares, supplies collateral to Moolah, and then refunds any unused input back to msg.sender. For the wrapped-native token, _refund() unwraps and pays the refund via a low-level native call rather than an ERC-20 safeTransfer. <pre> JavaScript uint256 refund0 = _amount0Desired - amount0Used; uint256 refund1 = _amount1Desired - amount1Used; if (refund0 > 0) _refund(TOKEN0, refund0, msg.sender); if (refund1 > 0) _refund(TOKEN1, refund1, msg.sender); </pre> <pre> JavaScript function _refund(address token, uint256 amount, address to) internal { if (token == WRAPPED_NATIVE) { (bool ok,) = payable(to).call{ value: amount }(""); if (!ok) revert BnbTransferFailed(); } else { IERC20(token).safeTransfer(to, amount); } } </pre>

A refund on the wrapped-native leg is expected whenever the supplied two-token amounts do not match the V3 position's required ratio. This is normal concentrated-liquidity behavior.

Impact

If msg.sender is a contract that cannot accept native coin (no payable receive() / fallback()), or an implementation that reverts on unexpected native transfers), the refund call fails and the entire deposit() reverts with BnbTransferFailed().

Because the refund is the last step, the transaction rolls back atomically: no shares are left minted and no funds are stolen.

- EOAs — unaffected
- Contracts — deposit via msg.value may fail when a wrapped-native refund is due; use wrapped-native ERC-20 instead

Recommendation

Refund the wrapped-native leg as ERC-20 instead of unwrapping to native coin, or add a refundReceiver parameter. At minimum, document that contract callers must not use msg.value unless they can accept native coin refunds.

Status

Acknowledged, no fix implemented, it is presumed that if a native refund is desired, the user performing a deposit must be able to receive such funds.

[L04] Permissionless market creation lacks safeguards for V3Provider collateral markets

Contract V3Provider.sol

Severity Low

Description

Moolah.createMarket is gated so that only OPERATOR accounts can create markets while an OPERATOR exists, and it opens to anyone if the OPERATOR role is emptied (getRoleMemberCount(OPERATOR) == 0). Its only validation is that the IRM and LLTV are enabled, that the tokens and oracle are non-zero, and two peek sanity calls whose return values are ignored. Several impacts arise from this when the collateral token is a V3Provider share.

Description

Impact

1. There is no binding between V3Provider collateral and its canonical oracle. The creator chooses MarketParams.oracle freely, and nothing requires a market that uses V3Provider shares as collateral to use V3ProviderOracle. The peek sanity calls confirm only that the oracle does not revert, not that it prices the collateral correctly, so a wrong or attacker-controlled oracle passes. If the OPERATOR gate is ever relaxed, or if an OPERATOR pairs the wrong oracle by mistake, a market can pair V3Provider shares with an oracle that overvalues them, letting

an attacker supply a small amount of collateral, borrow heavily against the inflated valuation, and drain that market's loan liquidity.

2. Provider registration is decoupled from market creation. The liquidation hook in Moolah.liquidate runs only when providers[id][marketParams.collateralToken] != address(0), and registration is done by setProvider, which is MANAGER gated and separate from the OPERATOR gated createMarket. Splitting these across two roles and two transactions lets a V3Provider market go live in an inconsistent state. If the provider is never registered, deposits and withdrawals still sync slisBNBx because the provider calls _afterCollateralChange directly, but liquidations skip the hook, so a liquidated borrower's slisBNBx is not re-synced or burned until their next interaction. If the wrong provider is registered, the hook calls that provider's liquidate, which reverts through if (MOOLAH.idToMarketParams(id).collateralToken != address(this)) revert InvalidMarket(), and that revert propagates and reverts the whole liquidation, so positions in that market become unliquidatable and the protocol accrues bad debt.
3. A V3Provider market can be created while the share price is 0. When the vault is empty (totalSupply == 0)ns 0, and createMarketcalls peek but ignores the return, so no price floor is enforced at creation. This is transient and self-resolves once deposits arrive, but the sanity check provides no protection here.
4. createMarket permits V3Provider shares to be set as the loan token. Borrowing would transfer them since Moolah is the caller and the token's transfer restriction allows Moolah, searchable by the borrower and redeemable.

Recommendation Consider ensuring that market creation is never permissionless, wherein the OPERATOR role is always assigned to a trusted ListaDao address.

Status Acknowledged, it is presumed that market creation will never be permissionless i.e. there will always be a trusted OPERATOR role creating moolah markets.

[L05] Dust vault state bricks all deposits through totalValueBefore rounding to zero

Contract V3Provider.sol

Severity Low

Description
In V3Provider.deposit, when shares already exist the code values the existing position and reverts if that value is zero:

```
JavaScript
uint256 totalValueBefore;
```

```

uint256 supplyBefore = totalSupply();
uint160 fairSqrtPriceX96 =
IV3DexAdapter(ADAPTER).fairSqrtPriceX96();
if (supplyBefore > 0) {
    (uint256 total0Before, uint256 total1Before) =
IV3DexAdapter(ADAPTER).positionAmountsAt(fairSqrtPriceX96);
    totalValueBefore = _amountsValueUsd(total0Before, total1Before);
    if (totalValueBefore == 0) revert ZeroShares();
}

```

The ZeroShares revert guards the division on the following line, but it turns a would-be division by zero into a deposit denial of service: while `supplyBefore > 0` and `totalValueBefore == 0`, every deposit reverts.

`_amountsValueUsd` computes $(\text{amount0} * \text{price0}) / (10 ** \text{DECIMALS0}) + (\text{amount1} * \text{price1}) / (10 ** \text{DECIMALS1})$ using integer division that rounds down. For an 18-decimal token priced near 1 USD (`price == 1e8`), the term $(\text{amount0} * 1e8) / 1e18$ returns 0 for any `amount0 < 1e10` wei, which is under about 0.00000001 USD. So if the adapter position holds only sub-dust on both legs, `totalValueBefore` rounds to 0.

JavaScript

```

/// @dev Value token0/token1 amounts as 8-decimal USD through the
resilient oracle.
function _amountsValueUsd(uint256 amount0, uint256 amount1) internal
view returns (uint256) {
    uint256 price0 = IOracle(resilientOracle).peek(TOKEN0); // 8
decimals
    uint256 price1 = IOracle(resilientOracle).peek(TOKEN1); // 8
decimals
    return (amount0 * price0) / (10 ** DECIMALS0) + (amount1 * price1) /
(10 ** DECIMALS1);
}

```

Impact

This state is reachable. Withdrawals are proportional, so the position value and the share supply shrink together, and the vault can be drained until the absolute position value is sub-dust while a few wei of shares remain. A full redemption where `shares == totalSupply` instead drops supply to 0 and reaches the safe seed branch, so the dust state requires a deliberate leave-dust redemption, meaning the sole or last holder redeems all but a few wei. An attacker who is the only holder can trigger this cheaply by locking a few wei of shares, which then blocks all new deposits. Only deposits are affected, since `redeemShares` and `withdraw` have no `totalValueBefore` check, so existing holders can still exit and no funds are locked.

	The vault is recoverable, so this is not a permanent brick. The dust holder can redeem the remaining wei to bring supply to 0, which restores the seed branch, or anyone can donate liquidity directly to the adapter position through the NonfungiblePositionManager increaseLiquidity, which has no owner check, pushing totalValueBefore above zero. Both recoveries are out of band and reactive, and the donation path transfers value to the dust holders, so relying on either is fragile.
Recommendation	Consider maintaining a permanent protocol seed deposit that is never withdrawn, but depends on the protocol never removing it. Alternatively, the permissionless increaseLiquidity via the NPM contract can be used as well to always ensure a sufficient protocol seed within a V3Provider market.
Status	Acknowledged, the check has been refactored but remain. ListaDao has indicated that they will be the first depositor to seed an initial amount of shares that ensures that current V3DexAdapter position will never rounddown to zero to cause deposit reverts.

[L06] Provider to SlisBNBxMinter wiring has no atomicity or consistency check,

Contract	SlisBNBV3Provider.sol & V3Provider.sol
Severity	Low
Description	Description The link between SlisBNBV3Provider and SlisBNBxMinter is controlled by two independent switches that live in different contracts and are set by different functions: setSlisBNBxMinter on the provider (which populates slisBNBxMinter) and addModule on the minter (which registers the provider as a module). Nothing enforces that both are set together or in a specific order, and each half behaves incorrectly when the other is missing. <pre> JavaScript /// @dev Value token0/token1 amounts as 8-decimal USD through the resilient oracle. function _amountsValueUsd(uint256 amount0, uint256 amount1) internal view returns (uint256) { uint256 price0 = IOracle(resilientOracle).peek(TOKEN0); // 8 decimals uint256 price1 = IOracle(resilientOracle).peek(TOKEN1); // 8 decimals return (amount0 * price0) / (10 ** DECIMALS0) + (amount1 * price1) / (10 ** DECIMALS1); } </pre> Impact 1. In the first scenario, If the module is added on the minter and the market deployed and starts accepting deposits, but setSlisBNBxMinter is not yet called on the provider, in _syncPosition, the provider always updates

userTotalDeposit, but it only calls minter.rebalance when slisBNBxMinter != address(0).

- a. While unwired, deposits and withdrawals keep userTotalDeposit accurate but never push anything to the minter. At the same time the module is live in the minter, and syncUserModuleLp is permissionless: its only check is that the module is registered.
- b. Anyone can call it for any account, and it reads the provider's getUserBalanceInBnb (which works while unwired because it only needs userTotalDeposit plus the adapter and oracle) and mints or burns slisBNBx through _rebalanceUserLp.
- c. The result is that a user can deposit collateral, have someone call syncUserModuleLp to mint the discounted slisBNBx to the delegatee plus the fee part to the MPC wallets, then withdraw the backing collateral through the provider. Because the provider is unwired, the withdrawal never calls minter.rebalance, so nothing is burned. The minted slisBNBx is only reversed if someone later calls syncUserModuleLp again, allowing possible snapshot of undeserved BNB launchpool rewards

2. In the second scenario, setSlisBNBxMinter set on the provider but the provider not registered on the minter via addModule.
 - a. Every _syncPosition calls minter.rebalance, which requires moduleConfig[msg.sender].moduleAddress == msg.sender. For an unregistered provider this is address(0) != provider, so the call reverts with "unauthorized module". Since _syncPosition makes this call with no try/catch, it bricks every deposit, withdrawal, and liquidation on the provider, leaving funds stuck and positions unliquidatable.

Recommendation

Ensure that the provider and minter wiring as a single atomic configuration step. Register the module on the minter and call setSlisBNBxMinter on the provider together, before the market accepts deposits, and never leave either half set without the other.

Status

Acknowledged, it is presumed that appropriate deployment steps are in place to ensure that every V3Provider created market must have the slisBNBxMinter module registration and setting performed atomically as required.

[I01] getTotalAmounts is spot priced and must not be used for valuation

Contract V3Provider.sol

Severity **Informational**

Description

Description

The V3Provider.getTotalAmounts() function reports the position composition by reading positionAmountsAt(spotSqrtPriceX96()), which derives its price from the pool's slot0. That spot price can be manipulated within a single transaction, for example through a flash loan that moves the pool, so the amounts returned by this function can be skewed away from their true value.

JavaScript

```
/// @inheritdoc IV3Provider
function getTotalAmounts() public view returns (uint256 total0,
uint256 total1) {
    return
    IV3DexAdapter(ADAPTER).positionAmountsAt(IV3DexAdapter(ADAPTER).spotSqrt
    PriceX96());
}
```

Impact

The function is documented as being intended for display and bot use only, and no part of the valuation path relies on it. Collateral valuation instead uses fairSqrtPriceX96, which is manipulation resistant. The concern is for external integrators, wherein any third party that reads getTotalAmounts() and treats it as a trustworthy valuation source would be exposed to manipulated figures and could make incorrect pricing or collateral decisions based on them.

Recommendation	Consider clearly documenting that it is spot priced and manipulable. Direct integrators who need a valuation to use the fair price view based on fairSqrtPriceX96 instead.
Status	Acknowledged, the IV3Provider has sufficient documentation in place indicating that getTotalAmounts returns total token0/token1 backing the vault at the current pool spot.

[I02] Inherited ERC-4626 preview, convert, and max view functions do not reflect the V3Provider share pricing

Contract	V3Provider.sol
Severity	Informational

Description	Description V3Provider is a two-token V3 LP vault that issues ERC-4626 shares, but it disables the four standard mutating ERC-4626 entrypoints. The functions deposit(uint256,address), mint(uint256,address), withdraw(uint256,address,address), and redeem(uint256,address,address) all revert with StandardEntryDisabled, because deposits and redemptions are handled by custom two-token functions that compute shares from the USD value of the underlying position rather than from the inherited ERC-4626 conversion math.
--------------------	---

JavaScript

```
/// @dev Single-asset ERC-4626 entry is disabled - this is a two-token
LP vault. Use the two-token
```

```

    ///      deposit(marketParams,...) / withdraw(marketParams,...) /
redeemShares().
    function deposit(uint256, address) public pure override returns
(uint256) {
        revert StandardEntryDisabled();
    }

    function mint(uint256, address) public pure override returns (uint256)
{
        revert StandardEntryDisabled();
    }

    function withdraw(uint256, address, address) public pure override
returns (uint256) {
        revert StandardEntryDisabled();
    }

    function redeem(uint256, address, address) public pure override
returns (uint256) {
        revert StandardEntryDisabled();
    }

```

The contract does not override the inherited view and quote surface, so `previewDeposit`, `previewMint`, `previewWithdraw`, `previewRedeem`, `convertToShares`, `convertToAssets`, `maxDeposit`, `maxMint`, `maxWithdraw`, and `maxRedeem` remain the inherited OpenZeppelin ERC4626Upgradeable implementations using the V3Provider overlying `totalSupply` and `totalAssets` functions.

JavaScript

```

/** @dev See {IERC4626-convertToShares}. */
function convertToShares(uint256 assets) public view virtual returns
(uint256) {
    return _convertToShares(assets, Math.Rounding.Floor);
}

/** @dev See {IERC4626-convertToAssets}. */
function convertToAssets(uint256 shares) public view virtual returns
(uint256) {
    return _convertToAssets(shares, Math.Rounding.Floor);
}

/** @dev See {IERC4626-maxDeposit}. */

```

```

function maxDeposit(address) public view virtual returns (uint256) {
    return type(uint256).max;
}

/** @dev See {IERC4626-maxMint}. */
function maxMint(address) public view virtual returns (uint256) {
    return type(uint256).max;
}

/** @dev See {IERC4626-maxWithdraw}. */
function maxWithdraw(address owner) public view virtual returns
(uint256) {
    return _convertToAssets(balanceOf(owner), Math.Rounding.Floor);
}

/** @dev See {IERC4626-maxRedeem}. */
function maxRedeem(address owner) public view virtual returns
(uint256) {
    return balanceOf(owner);
}

/** @dev See {IERC4626-previewDeposit}. */
function previewDeposit(uint256 assets) public view virtual returns
(uint256) {
    return _convertToShares(assets, Math.Rounding.Floor);
}

/** @dev See {IERC4626-previewMint}. */
function previewMint(uint256 shares) public view virtual returns
(uint256) {
    return _convertToAssets(shares, Math.Rounding.Ceil);
}

/** @dev See {IERC4626-previewWithdraw}. */
function previewWithdraw(uint256 assets) public view virtual returns
(uint256) {
    return _convertToShares(assets, Math.Rounding.Ceil);
}

/** @dev See {IERC4626-previewRedeem}. */
function previewRedeem(uint256 shares) public view virtual returns
(uint256) {
    return _convertToAssets(shares, Math.Rounding.Floor);
}

```

Impact

This produces three problems for external consumers.

1. First, real redemption returns two token amounts (token0 and token1), while previewRedeem and maxRedeem return a single accounting-asset number, so the quoted value does not match what a redeemer receives.
2. Second, the [ERC-4626 standard](#) requires that the preview functions stay consistent with their corresponding actions, yet here deposit, mint, withdraw, and redeem always revert while their previews return non-reverting numbers, and maxDeposit and maxMint advertise `type(uint256).max` for entrypoints that always revert. A standards-compliant ERC-4626 router can read maxDeposit as effectively unlimited and a positive previewDeposit, call deposit, and then revert.
3. Third, maxWithdraw and maxRedeem are derived from balanceOf(owner), but shares are non-transferable except by Moolah and normally held in Moolah as collateral, so for a typical end-user address these return 0 and misrepresent the actual redeemable position.
4. The inherited convertToShares and convertToAssets also diverge from the contract's true share pricing, which compounds the problem. On the deposit side, the actual mint computes $\text{shares} = \text{addedValue} * \text{supplyBefore} / \text{totalValueBefore}$ as a ratio of 8-decimal USD values at the fair price. This is algebraically the same ratio as convertToShares once the accounting-asset price factor cancels, but it still differs in three ways
 - a. The inherited formula adds the + 1 virtual offsets to supply and assets while the mint path uses the raw ratio with a separate first-deposit branch, the inherited path applies an extra flooring step because a consumer must first convert value into accounting-asset units and then convert again while the mint path floors only once, and most importantly convertToShares takes a single scalar with no notion of the two-token ratio.
 - b. The mint values only the liquidity actually added, which excludes the portion refunded because a V3 position consumes the two tokens in the range's required ratio, so convertToShares over-estimates shares whenever the supplied amounts are off-ratio.
5. convertToAssets derives from totalAssets, which values the position at the fair (rate-implied) price, but removeLiquidity settles the withdrawn liquidity against the live pool price, and the matching custom quote previewRemoveLiquidity correctly uses the spot price rather than the fair price. When spot and fair differ, the realized output and its value differ from what convertToAssets reports. In addition, removeLiquidity returns two token amounts made up of the share-proportional fraction of liquidity at spot plus the share-proportional fraction of idle inventory, while convertToAssets collapses this into a single accounting-asset number that is a fraction of the fair-value position NAV plus

the + 1 virtual offsets. The two only coincide in value at the fair price, so convertToAssets is not a valid redemption quote.

There is no impact on core contract logic. None of the contract's own state-changing paths call these inherited views. Share minting and burning use custom USD-proportional math and the ERC-20 _mint and _burn primitives rather than the ERC-4626 _deposit and _withdraw workflows, the dangerous _deposit and _withdraw paths are unreachable because their entry points revert, and Moolah prices the collateral through a separate oracle rather than through convertToAssets. The impact is therefore confined to off-chain integrators, user interfaces, and standards-compliant routers that read the ERC-4626 interface in good faith and receive misleading values.

Recommendation

Consider override the previewDeposit, previewMint, previewWithdraw, previewRedeem, convertToShares, convertToAssets, maxDeposit, maxMint, maxWithdraw, and maxRedeem revert, or return 0 where reverting is not appropriate, so that integrators cannot read a usable quote from a path that always reverts or from conversion math that does not match the real two-token, fair-versus-spot pricing. Direct integrators to the custom two-token quote functions previewDepositAmounts and previewRemoveLiquidity, and document that this vault deliberately deviates from the single-asset ERC-4626 quote semantics.

Status

Acknowledged, documentation is in place to direct users to use V3Provider market actions.

[I03] MPC wallet capacity can cause slisBNBx rebalance reverts that propagates to market actions

Contract SlisBNBV3Provider.sol & V3Provider.sol

Severity **Informational**

Description

The SlisBNBV3Provider._syncPosition() function calls the slisBNBx minter's rebalance routine to keep a user's slisBNBx balance aligned with their collateral. When the rebalance mints the protocol fee portion of slisBNBx to the MPC fee wallets, the mint fails if those wallets lack enough remaining capacity under their configured caps, and the rebalance reverts.

Description

```
JavaScript
function _syncPosition(Id id, address account) internal {
    uint256 current = MOOLAH.position(id, account).collateral;
    if (current >= userMarketDeposit[account][id]) {
        userTotalDeposit[account] += current -
userMarketDeposit[account][id];
    } else {
        userTotalDeposit[account] -= userMarketDeposit[account][id] -
current;
    }
    userMarketDeposit[account][id] = current;
}
```

```

if (slisBNBxMinter != address(0)) {
    ISlisBNBxMinter(slisBNBxMinter).rebalance(account);
}
}

```

JavaScript

```

function _mintToMPCs(uint256 amount) internal {
    uint256 leftToMint = amount;
    // loop through the MPC wallets
    for (uint256 i = 0; i < mpcWallets.length; ++i) {
        // mint completed
        if (leftToMint == 0) break;
        // get the current wallet
        MPCWallet storage wallet = mpcWallets[i];
        // get slisBNBx balance
        uint256 balance = wallet.balance;
        // balance not reached the cap yet
        if (balance < wallet.cap) {
            uint256 toMint = balance + leftToMint > wallet.cap ? wallet.cap
- balance : leftToMint;
            // mint slisBNBx to the wallet
            SLISBNB_X.mint(wallet.walletAddress, toMint);
            // add up balance
            wallet.balance += toMint;
            // deduct leftToMint
            leftToMint -= toMint;
        }
    }

    require(leftToMint == 0, ErrorsLib.EXCEED_MPC_CAP);
}

```

Impact

This revert is not contained. `_syncPosition` runs inside `_afterCollateralChange`, so when the `rebalance` reverts, `_afterCollateralChange` reverts, and every `V3Provider` action that calls it reverts in turn. The affected upstream actions are the full set of position changing flows:

- Deposits revert, so users cannot add liquidity and receive shares supplied as collateral.
- Withdrawals revert, so users cannot remove liquidity and pull their underlying tokens back.
- Share withdrawals revert, so users cannot move their collateral shares out to a chosen receiver.

	• Share supplies revert, so users cannot post existing shares as collateral. • The liquidation callback reverts, which is the most damaging case. Because the provider's liquidation hook runs through the same sync path, a revert there blocks liquidation of an unhealthy position entirely. An affected borrower becomes unliquidatable, and the resulting unpaid debt can turn into bad debt for the market. The available headroom in the MPC fee wallets therefore becomes a hard dependency for core protocol operations rather than just the reward accounting. Once the wallets fill up, normal user actions and, more seriously, liquidations cannot proceed until MPC capacity is increased or freed.
Recommendation	Consider ensuring that the MPC fee wallets has enough capacity for expected minting and monitor their usage so that caps are raised or additional wallets are added before they are exhausted.
Status	[Open / Fixed / Acknowledged]

[I04] deposit silently overrides the desired amount for the native leg instead of validating it against msg.value	
Contract	V3Provider.sol
Severity	Informational
Description	Description When V3Provider.deposit() receives native tokens, meaning when msg.value is greater than zero, it wraps that value and assigns it as the desired amount for whichever pool leg is the wrapped native token. In doing so it overwrites the caller supplied desired amount for that leg with msg.value and never checks that the two agree. If a caller passes a desired amount for the native leg that differs from msg.value, the supplied desired amount is silently ignored and only msg.value is used. Impact This creates a mismatch between what the caller expressed and what the function acts on. The caller's stated input is discarded without any error, which can hide mistakes in how the deposit was constructed and make the outcome harder to reason about. There is no loss of funds, but the lack of validation weakens input safety and can lead to confusion when the amount that is actually deposited does not match the amount the caller thought they specified.
Recommendation	When msg.value is greater than zero, add a check that the caller supplied desired amount for the wrapped native leg equals msg.value, for example require that amount0Desired == msg.value when the native leg is token0, and the equivalent check for token1. Revert on any mismatch so that the caller's declared amount and the sent native value must agree, rather than silently overriding the declared amount.

Status	Acknowledged, no fix implemented. It is presumed that users would not incorrectly set mismatching msg.value and desired amounts.
---------------	--

[I05] Redundant _approve in deposit and supplyShares since transferFrom is caller gated and ignores allowance

Contract	V3Provider.sol
Severity	Informational
Description	Description In deposit and supplyShares, the provider mints or receives the share tokens to itself and then calls _approve(address(this), address(MOOLAH), shares) before invoking MOOLAH.supplyCollateral. Moolah pulls the collateral from the provider using safeTransferFrom(provider, MOOLAH, shares) with msg.sender equal to Moolah. <pre>JavaScript _approve(address(this), address(MOOLAH), shares);</pre> The _approve call is redundant, because the contract overrides transferFrom so that it does not consult or spend any allowance. The override only checks msg.sender == MOOLAH and then calls _transfer directly, without invoking _spendAllowance. As a result, the transfer initiated by supplyCollateral succeeds purely on the caller check, and the allowance written by _approve is never read or decremented. The allowance slot is simply overwritten with the latest share amount on each call and left as a stale value. Impact The impact is limited to a wasted storage write on every deposit and supplyShares call, plus reduced code clarity, because the code implies an allowance based pull that does not reflect how the restricted token actually transfers. There is no security impact, since the msg.sender == MOOLAH check is the real access control on the transfer.
Recommendation	Consider removing the _approve call from deposit and supplyShares, since transferFrom is caller gated and ignores allowance, which saves one storage write per call. Add a short comment noting that transferFrom is gated on msg.sender == MOOLAH and needs no allowance, so a future change that restores standard allowance checking in transferFrom also restores the approval.
Status	Fixed, now includes the _spendAllowance call within transferFrom which subtracts the approved allowance to the Moolah contract by the V3Provider. Note that this is optional but can be kept as a safeguard to ensure the Moolah contract does not have excess allowance of V3Provider shares (although theoretically it is infinite).

[I06] Settable but non functional approve creates ghost approvals and a latent upgrade risk

Contract	V3Provider.sol
Severity	Informational
Description	Description The contract overrides transferFrom so that it is gated on msg.sender == address(MOOLAH) and transfers directly without spending any allowance. It does not override approve, which remains the standard functional ERC20 implementation. A holder of wallet held shares, for example shares obtained through withdrawShares, can therefore call approve(spender, value): the call succeeds, sets the allowance, and emits an Approval event. However the approved spender can never move the shares, because transferFrom reverts with OnlyMoolah for any caller other than Moolah. The result is a ghost approval that exists in storage and in events but grants no power. <pre>JavaScript _approve(address(this), address(MOOLAH), shares);</pre>
Description	Impact There is no direct fund safety risk today. Approval based phishing on these shares is ineffective, and no one can drain a holder through an allowance, since only Moolah can move shares at all. Two real risks remain. 1. First, composability and user experience. Because approve succeeds and allowance returns a non zero value, users and integrating contracts such as DEX routers, wrappers, bridges, and aggregators will believe that an approve followed by transferFrom will work, when in fact it silently reverts with OnlyMoolah. Off chain systems that track allowances will display live looking approvals that do nothing. 2. Second, a latent upgrade risk. All of these contracts are UUPS upgradeable, and every historical approve persists as a real allowance in storage. If a future upgrade ever relaxes transferFrom to standard allowance based transfers, for example to make the shares tradable, all accumulated ghost approvals activate retroactively and become exploitable, and users will not know to revoke approvals they believed were inert.
Recommendation	Consider overriding approve to revert, mirroring how the standard ERC4626 single asset entry functions are already disabled with StandardEntryDisabled. This ensures no allowance can ever be set, so there are no ghost approvals to mislead integrations and none that could activate on a future upgrade.
Status	Acknowledged, no fix implemented. There is no current risk of approval considering transfers of V3Provider shares can only be performed by the Moolah contract.

[I07] Missing empty string validation for _name and _symbol in __V3Provider_init

Contract V3Provider.sol

Severity Informational

Description

The __V3Provider_init initializer validates that its address parameters are non zero, but it does not verify that the _name and _symbol string parameters are non empty. As a result, the ERC4626 share token can be initialized with an empty name or an empty symbol.

Description

JavaScript

```
function __V3Provider_init(
    address _admin,
    address _manager,
    address _bot,
    address _resilientOracle,
    address _accountingAsset,
    string calldata _name,
    string calldata _symbol
) internal onlyInitializing {
    if (
        _admin == address(0) ||
        _manager == address(0) ||
        _bot == address(0) ||
        _resilientOracle == address(0) ||
        _accountingAsset == address(0)
    ) {
        revert ZeroAddress();
    }

    __ERC20_init(_name, _symbol);
    __ERC4626_init(IERC20(_accountingAsset));
    __AccessControl_init();
    __ReentrancyGuard_init();

    _grantRole(DEFAULT_ADMIN_ROLE, _admin);
    _grantRole(MANAGER, _manager);
    _setRoleAdmin(BOT, MANAGER);
    _grantRole(BOT, _bot);

    resilientOracle = _resilientOracle;
    accountingAssetDecimals =
IERC20Metadata(_accountingAsset).decimals();
}
```

Impact

	These values populate the ERC20 metadata of the vLP share token that serves as Moolah collateral. If either is left empty, the token presents blank identifiers on block explorers, wallets, and any integrating interface, which can cause confusion and display or integration problems. There is no direct security impact, but because initialization happens only once, an empty value cannot be corrected without a contract upgrade.
Recommendation	Consider validating that <code>_name</code> and <code>_symbol</code> in <code>__V3Provider_init</code> and reject empty values, for example by requiring that the byte length of each string is greater than 0, reverting with a clear error when either is empty.
Status	Acknowledged, no fix implemented, it is presumed that the deployer will perform necessary input checks for the name/symbol before a V3Provider market deployment.

[I08] MPC wallet capacity reverts in `_syncPosition` can block core V3Provider actions including liquidation

Contract	V3Provider.sol
Severity	Informational
Description	Description The <code>SlisBNBV3Provider._syncPosition()</code> function calls the <code>slisBNBx</code> minter's rebalance routine as part of keeping a user's <code>slisBNBx</code> balance aligned with their collateral. That rebalance can revert when the MPC fee wallets lack sufficient capacity to receive the fee portion of <code>slisBNBx</code> being minted. Because <code>_syncPosition</code> is invoked through <code>_afterCollateralChange</code>, this revert is not contained and propagates upward, causing the calling V3Provider action to fail entirely. <pre> JavaScript function _afterCollateralChange(Id id, address account) internal override { _syncPosition(id, account); } function _syncPosition(Id id, address account) internal { // V3Provider token != slisBNB or WBNB token uint256 current = MOOLAH.position(id, account).collateral; if (current >= userMarketDeposit[account][id]) { userTotalDeposit[account] += current - userMarketDeposit[account][id]; } else { userTotalDeposit[account] -= userMarketDeposit[account][id] - current; } userMarketDeposit[account][id] = current; if (slisBNBxMinter != address(0)) { ISlisBNBxMinter(slisBNBxMinter).rebalance(account); } } </pre>

```

    }
}

```

JavaScript

```

function _mintToMPCs(uint256 amount) internal {
    uint256 leftToMint = amount;
    // loop through the MPC wallets
    for (uint256 i = 0; i < mpcWallets.length; ++i) {
        // mint completed
        if (leftToMint == 0) break;
        // get the current wallet
        MPCWallet storage wallet = mpcWallets[i];
        // get slisBNBx balance
        uint256 balance = wallet.balance;
        // balance not reached the cap yet
        if (balance < wallet.cap) {
            uint256 toMint = balance + leftToMint > wallet.cap ? wallet.cap
- balance : leftToMint;
            // mint slisBNBx to the wallet
            SLISBNB_X.mint(wallet.walletAddress, toMint);
            // add up balance
            wallet.balance += toMint;
            // deduct leftToMint
            leftToMint -= toMint;
        }
    }

    require(leftToMint == 0, ErrorsLib.EXCEED_MPC_CAP);
}

```

Impact

The affected actions are the core position changing flows: deposits, withdrawals, and share supply operations all revert when the rebalance reverts. The most serious case is liquidation, since the provider's liquidation path also runs through this same sync logic. A revert there blocks liquidation of an unhealthy position, which can leave a borrower unliquidatable and allow the resulting shortfall to turn into bad debt. In effect, the available capacity of the MPC fee wallets becomes a hard dependency for essential protocol operations rather than just the reward accounting, and once that capacity is exhausted these operations cannot proceed until it is increased or freed.

Recommendation

Consider validating that `_name` and `_symbol` in `__V3Provider_init` and reject empty values, for example by requiring that the byte length of each string is greater than 0, reverting with a clear error when either is empty.

Status

Acknowledged, it is presumed that sufficient MPC wallet capacity is in place to prevent slisBNBx rebalancing from reverting market actions via V3Provider.

[I09] BnbTransferFailed error name is chain specific and misleading on non-BSC deployments

Contract V3Provider.sol & V3DexAdapter.sol

Severity **Informational**

Description

Both V3Provider and V3DexAdapter declare and revert with an error named BnbTransferFailed when a native coin transfer fails.

- In V3Provider this happens in the refund path for the wrapped native leg
- in V3DexAdapter it happens in the send path that unwraps the wrapped native token and forwards the native coin.

Description

```
JavaScript  
error BnbTransferFailed();
```

Impact

These two contracts are the chain agnostic base contracts, and their own documentation states that the wrapped native token is WBNB on BSC and WETH on Ethereum. The Ethereum subclasses, WstETHV3DexAdapter and WbETHV3DexAdapter along with their providers, deploy against WETH, where the native coin is ETH rather than BNB. On those deployments a failed native transfer reverts with BnbTransferFailed, which names the wrong asset. The behavior is correct, but the revert reason is misleading for anyone reading a failed Ethereum transaction, since the failure has nothing to do with BNB.

Recommendation Consider renaming the error to a chain neutral name such as NativeTransferFailed in both V3Provider and V3DexAdapter, and update the revert sites accordingly.

Status Acknowledged, no fix implemented. This is purely a naming/cosmetic issue so a fix is optional.

[I10] Hook bearing token0 or token1 would reintroduce the reentrancy through the adapter to vault refund inside addLiquidity

Contract V3Provider.sol & V3DexAdapter.sol

Severity **Informational**

Description

The reentrancy fix within deposit moved only the depositor refund to after _mint and supplyCollateral, following the checks effects interactions pattern. A separate, earlier refund still executes mid deposit.

When `V3Provider.deposit()` forwards the input tokens to the adapter and calls `V3DexAdapter.addLiquidity()` with `refundTo` set to the vault itself, the adapter returns the unused input leg to the vault through `_sendToken()` while still inside `addLiquidity()`. At that point the liquidity has already been added, so the adapter's NAV already includes it, but the vault has not yet minted shares, so `totalSupply` is still the old value. The share oracle therefore reads an inflated price, since $\text{peek}(\text{share}) = \text{totalValue} * 10^{\text{dec}} / \text{supply}$ uses an increased `totalValue` over a stale supply.

JavaScript

```
// Refund unused input to the depositor now that shares are minted
// and supplied (CEI): NAV and
// totalSupply are consistent, so the (native-BNB) refund callback
// cannot inflate the share price.
uint256 refund0 = _amount0Desired - amount0Used;
uint256 refund1 = _amount1Desired - amount1Used;
if (refund0 > 0) _refund(TOKEN0, refund0, msg.sender);
if (refund1 > 0) _refund(TOKEN1, refund1, msg.sender);
```

Impact

If `token0` or `token1` passes control to attacker controlled code during that transfer, the attacker can reenter Moolah and borrow against the temporarily inflated share collateral price, which leaves bad debt once the price returns to normal. The `nonReentrant` guards on the provider and the adapter do not prevent this, because the deposit flow is not running inside a Moolah call at that moment, so Moolah's own guard is not held and the cross contract oracle read is not gated. This is the common path, because a ratio mismatch refunds one leg on essentially every deposit.

Currently, all live pairs, namely `slisBNB`, `WBNB`, `wstETH`, `WETH`, and `wbETH`, are hookless standard ERC20 tokens, so no exploit exists, any only affects token in the future with a hook bearing pair.

Recommendation	Consider requiring every future <code>token0</code> and <code>token1</code> pair to be a no callback ERC20 token, meaning no transfer hooks,
Status	Acknowledged, it is presumed that sufficient whitelist checks will be in place to not allow creating V3 LP Collateral markets with tokens of such behaviors.

V3DexAdapter

[M02] collectAndCompound only runs on user actions, leaving fees uncompounded on buy and hold vaults

Contract V3DexAdapter.sol

Severity Medium

Description

The V3DexAdapter compounding routine, collectAndCompound, reinvests earned trading fees back into the position so those fees also begin earning. This reinvestment is the main lever for yield in the vault. The routine only executes as a side effect of deposit, withdraw, and redeem operations, and there is no way to trigger it on its own given the onlyProvider modifier gate.

JavaScript

```
/// @inheritdoc IV3DexAdapter
function collectAndCompound() external onlyProvider nonReentrant {
    _collectAndCompound();
}
```

Description

Impact

In the event that user market actions are infrequent, earned fees sit uncollected and uncompounded for long stretches of time. During those periods the idle fees are not working, so the position earns less than it could. The missed upside is not a loss of principal, but it is a shortfall in the yield the position is capable of generating, and it grows the longer the vault goes without a deposit, withdraw, or redeem.

Recommendation

Consider a dedicated BOT only collectAndCompound function to compound independently of user activity, so fees can be reinvested on a regular schedule rather than only when a user happens to deposit, withdraw, or redeem.

Status

Fixed, now includes a permissionless compound function within the V3Provider contract, allowing a ListaDao managed keeper (BOT) to routinely compound fees/idle tokens into the V3DexAdapter NPM position, and no longer depends solely on user market actions.

That utilizes a fair V3DexAdapter_collectAndCompound that performs initial checks that the pool spot price is within the a admin set maxSpotDeviationBps, ensuring that fees are only allowed to be compounded if current spot price is within the allowed deviation from the fairSqrtPriceX96 via _spotWithinFair. This enforces that when liquidity addition of compounded fees/idle tokens is performed, it is performed at a pool spot price within an allowed safe deviation range, not at an attacker's manipulated pool price.

[M03] Slippage bounds in removeLiquidity() exclude the pro-rata idle payout, causing possible reverts

Contract V3DexAdapter.sol

Severity **Medium**

Description

The V3DexAdapter.removeLiquidity() function forwards the caller's minAmount0 and minAmount1 slippage bounds into decreaseLiquidity(), where the NonfungiblePositionManager enforces them only against the amounts released by burning the removed liquidity. After that check, the function adds each token's pro-rata idle inventory, computed as $\text{idleToken} * \text{shares} / \text{totalShares}$, to the amounts sent to the receiver, and this idle portion is never compared against the caller's minimum amounts.

Description

JavaScript

```
/// @inheritdoc IV3DexAdapter
function removeLiquidity(
    uint256 shares,
    uint256 totalShares,
    uint256 minAmount0,
    uint256 minAmount1,
    address receiver
) external onlyProvider nonReentrant returns (uint256 amount0, uint256
amount1) {
    uint128 totalLiq = _getPositionLiquidity();
    uint128 liquidityToRemove = totalShares == 0 ? 0 :
uint128((uint256(totalLiq) * shares) / totalShares);

    if (liquidityToRemove > 0) {
        V3PositionLib.decreaseLiquidity(POSITION_MANAGER, tokenId,
liquidityToRemove, minAmount0, minAmount1);
        (amount0, amount1) = V3PositionLib.collectAll(POSITION_MANAGER,
tokenId);
    }

    // Pro-rata idle inventory (finding C): redeem the same fraction of
idle as of liquidity.
    if (totalShares > 0) {
        uint256 idleOut0 = (idleToken0 * shares) / totalShares;
        uint256 idleOut1 = (idleToken1 * shares) / totalShares;
        if (idleOut0 > 0) {
            idleToken0 -= idleOut0;
            amount0 += idleOut0;
        }
        if (idleOut1 > 0) {
            idleToken1 -= idleOut1;
            amount1 += idleOut1;
        }
    }
}
```

```

        if (amount0 > 0) _sendToken(TOKEN0, amount0, payable(receiver));
        if (amount1 > 0) _sendToken(TOKEN1, amount1, payable(receiver));

        emit LiquidityRemoved(shares, totalShares, amount0, amount1,
            receiver);
    }

```

This creates a mismatch with `previewRemoveLiquidity()`, which is the function a caller uses to size their minimums and which returns the total of the liquidity portion plus the idle portion. When a caller quotes with `previewRemoveLiquidity()`

```

JavaScript
/// @inheritdoc IV3DexAdapter
function previewRemoveLiquidity(
    uint256 shares,
    uint256 totalShares
) external view returns (uint256 amount0, uint256 amount1) {
    if (totalShares == 0 || shares == 0) return (0, 0);
    uint128 liquidityToRemove =
uint128((uint256(_getPositionLiquidity()) * shares) / totalShares);
    (amount0, amount1) = LiquidityAmounts.getAmountsForLiquidity(
        spotSqrtPriceX96(),
        TickMath.getSqrtRatioAtTick(tickLower),
        TickMath.getSqrtRatioAtTick(tickUpper),
        liquidityToRemove
    );
    amount0 += (idleToken0 * shares) / totalShares;
    amount1 += (idleToken1 * shares) / totalShares;
}

```

Impact

In the event that the user utilizes the return values as the slippage to perform withdrawals via `withdraw` and `redeemShares` and the subsequent, liquidity removal, the overestimated slippages can cause liquidity removal reverts.

Recommendation

The fix is complex, considering view functions cannot incorporate state changes. Consider instead require an off-chain automated slippage computation or necessary documentation to ensure users set appropriate slippage when performing liquidity removals.

Status

Fixed, now introduce a `_pendingFees` function that incorporates uncompounded pending fees on the V3 pool side within `positionAmountsAt` and used in `previewRemoveLiquidity` to compute the total liquidity used to compute the proportional share of underlying tokens when previewing liquidity to obtain.

Note that the `spotSqrtPriceX96` is still utilized but this is now documented within the `previewRemoveLiquidity`.

[M04] Single-sided inventory during BOT rebalance reverts at pool.mint

Contract V3DexAdapter.sol

Severity Medium

Description

The `V3DexAdapter.rebalance()` function is driven by the BOT role and recenters the position around a rate-implied tick range before minting fresh liquidity through `V3PositionLib.mint()`, which forwards to the `NonfungiblePositionManager` and ultimately calls `pool.mint()`. The rebalance enters the mint branch whenever either `total0 > 0` or `total1 > 0`, so a single-sided inventory (one of `total0` or `total1` equal to 0) can reach the mint call. The mint amounts are passed with `amount0Min` and `amount1Min` set to 0, so the periphery slippage check cannot stop it.

JavaScript

```
if (total0 > 0 || total1 > 0) {
  (uint256 newTokenId, uint128 liquidity, uint256 used0, uint256
used1) = V3PositionLib.mint(
    POSITION_MANAGER,
    TOKEN0,
    TOKEN1,
    FEE,
    newTickLower,
    newTickUpper,
    total0,
    total1,
    0,
    0
  );
  tokenId = newTokenId;
  mintedLiquidity = liquidity;
  idleToken0 = total0 - used0;
  idleToken1 = total1 - used1;
```

Description

For a correlated pair such as `slisBNB/WBNB`, the range is always centered on the rate with a half-width of ± 100 BPS (1%), and the pool spot price normally sits inside the range. In that in-range case, `LiquidityAmounts.getLiquidityForAmounts()` computes liquidity as the minimum of the token0-derived and token1-derived amounts.

JavaScript

```
function getLiquidityForAmounts(
```

```

        uint160 sqrtRatioX96,
        uint160 sqrtRatioAX96,
        uint160 sqrtRatioBX96,
        uint256 amount0,
        uint256 amount1
    ) internal pure returns (uint128 liquidity) {
        if (sqrtRatioAX96 > sqrtRatioBX96) (sqrtRatioAX96,
sqrtRatioBX96) = (sqrtRatioBX96, sqrtRatioAX96);

        if (sqrtRatioX96 <= sqrtRatioAX96) {
            liquidity = getLiquidityForAmount0(sqrtRatioAX96,
sqrtRatioBX96, amount0);
        } else if (sqrtRatioX96 < sqrtRatioBX96) {
            uint128 liquidity0 = getLiquidityForAmount0(sqrtRatioX96,
sqrtRatioBX96, amount0);
            uint128 liquidity1 = getLiquidityForAmount1(sqrtRatioAX96,
sqrtRatioX96, amount1);

            liquidity = liquidity0 < liquidity1 ? liquidity0 :
liquidity1;
        } else {
            liquidity = getLiquidityForAmount1(sqrtRatioAX96,
sqrtRatioBX96, amount1);
        }
    }
}

```

If one side is 0, the minimum is 0, and ListaV3Pool.mint() reverts on its require(amount > 0) check. The whole rebalance transaction reverts before reaching the InsufficientLiquidityMinted check.

JavaScript

```

// @inheritdoc IListaV3PoolActions
// @dev noDelegateCall is applied indirectly via _modifyPosition
function mint(
    address recipient,
    int24 tickLower,
    int24 tickUpper,
    uint128 amount,
    bytes calldata data
) external override lock returns (uint256 amount0, uint256 amount1)
{
    require(amount > 0);
}

```

Impact

This is not an originally realistic intended path. The _convertToOptimalRatio() step exists so the BOT can convert skewed inventory into the two-sided ratio the in-range

position requires, and a correctly built swap always leaves both sides non-zero. However, the BOT can realistically trigger the revert in two ways.

1. First, `_convertToOptimalRatio()` allows empty `swapData`, which returns the totals unchanged, so recentering without a swap while the collected inventory is essentially one-sided (the common state right after a rate move larger than 1% that triggers the rebalance) leaves one side at 0.
2. Second, `SwapInventoryLib.swap()` caps `amountIn` to the available balance, so an `amountIn` greater than or equal to the full balance of the sell side drives that side to exactly 0. The result in both cases is a stuck rebalance that reverts, a liveness issue rather than a loss of funds.
3. Third, the one realistic single-sided case occurs when spot deviates more than 1% from the rate so the rate-centered range no longer contains spot, where a correctly chosen single token mints successfully. This can happen for more general token pairs that are not LST token pairs.
 - a. For example, during the Celsius / 3AC event, stETH traded at roughly a 5 to 6% discount to ETH on the secondary market (around 0.94 to 0.95 ETH), while Lido's `stEthPerToken()` accounting rate stayed at or above 1.0x and kept climbing.
 - i. `centerRate = stEthPerToken() ≈ unaffected` by the depeg, so the new range is `[rate·0.99, rate·1.01]`, anchored near the "fair" accounting value.
 - ii. `pool spot = market price`, -5 to 6% below that. So spot sits entirely below `tickLower`.
 - iii. With spot below the range, `getLiquidityForAmounts` takes the `sqrtPriceX96 <= sqrtRatioAX96` branch, which needs only the lower token (100% of one side). A correctly built single-sided mint of that token succeeds and is in fact the correct V3 composition for a range that is entirely above spot.

Recommendation

Before the mint call in `rebalance()`, detect whether the current spot price lies inside the target tick range, and when it does, require that both `total0 > 0` and `total1 > 0`, reverting with a descriptive custom error instead of relying on the pool's bare `require(amount > 0)`.

Status

Acknowledged, given depeg risks are accepted and the likely pairs deployed are LST pairs, it is unlikely that single sided liquidity is held and as such, the risk is accepted.

[M05] Hardcoded tick range widths in `_initialTickRange` are rigid, and center the fallback on manipulable spot

Contract	V3DexAdapter.sol
Severity	Medium
Description	Description The V3DexAdapter._initialTickRange logic derives liquidity ranges from two hardcoded constants, and both are inflexible in ways that matter across different pairs and market regimes. <pre> JavaScript function _initialTickRange(uint256 centerRate) internal view returns (int24 initialTickLower, int24 initialTickUpper) { int24 tickSpacing = IListaV3Pool(P00L).tickSpacing(); if (centerRate != 0) { (initialTickLower, initialTickUpper) = _tickRangeForRate(centerRate, tickSpacing); } else { (, int24 currentTick) = IV3PoolMinimal(P00L).slot0(); initialTickLower = _floorTick(currentTick - FALLBACK_HALF_RANGE_TICKS, tickSpacing); initialTickUpper = _ceilTick(currentTick + FALLBACK_HALF_RANGE_TICKS, tickSpacing); } if (initialTickLower >= initialTickUpper) { initialTickUpper = initialTickLower + tickSpacing; } } function _tickRangeForRate(uint256 centerRate, int24 tickSpacing) internal view returns (int24 initialTickLower, int24 initialTickUpper) { uint256 lowerRate = (centerRate * (BPS - INITIAL_RANGE_BPS)) / BPS; uint256 upperRate = (centerRate * (BPS + INITIAL_RANGE_BPS)) / BPS; initialTickLower = _floorTick(_tickAtSqrtRatio(_sqrtPriceX96FromRate(lowerRate)), tickSpacing); </pre>

```

        initialTickUpper =
        _ceilTick(_tickAtSqrtRatio(_sqrtPriceX96FromRate(upperRate)),
        tickSpacing);
    }

```

Impact

1. INITIAL_RANGE_BPS = 100 produces a rate implied range of [centerRate-0.99, centerRate+1.01], meaning $\pm 1\%$ in price, which is approximately ± 100 ticks since $1.0001 \cdot 100 \approx 1.01$. For an LST pair with a healthy peg, $\pm 1\%$ is a reasonable and capital efficient concentration that maximizes fee capture near the rate. However, it is not wide enough to keep the position in range through a depeg or volatility spike. stETH traded roughly 5% to 6% below ETH in June 2022, which is 5 to 6 times this width. This compounds with the rebalance behavior: rebalance recenters on the rate rather than on market spot, so during a depeg, the market spot sits well outside [rate-0.99, rate+1.01], and a rebalance mints a range that does not contain spot. The new position is immediately single sided (100% LST) and out of range, so the rebalance actively fights the market.
2. FALLBACK_HALF_RANGE_TICKS = 500 sets ± 500 ticks around the spot tick for pure TWAP pairs, which is about $+5.1\%$ / -4.9% since $1.0001 \cdot 500 \approx 1.0513$. A single fixed width cannot fit general pairs.
 - a. It is far too wide for stablecoin pairs, causing near total capital inefficiency, and
 - b. It is far too narrow for volatile coins that move $> 5\%$ intraday, causing the position to be perpetually out of range with constant rebalancing.
 - c. It only fits a narrow band of medium volatility pairs, An additional concern is that despite the pure TWAP label, the range center for these pairs is the manipulable spot tick read at rebalance time, so an attacker able to move spot in the rebalance block shifts the entire minted range. This path is gated (BOT only, deadline, minLiquidity), so it is a grieving and sandwich risk rather than direct theft, but centering a range on raw spot is weaker than centering on the pool TWAP.

Note that all three deployed supported LST markets override `_lstNativeRate()` to return a non zero rate for their constructor pinned pair, so `rate == 0` never occurs and FALLBACK_HALF_RANGE_TICKS is never reached in the in scope deployments

Consider:

Recommendation

- Allowing both range widths to be tick spacing aligned, manager settable storage variables with hard min and max caps, mirroring how

centerRateThresholdBps and maxTwapDeviationBps are already handled, and default the LST width to 100 bps.

- For the general and TWAP path, center the range on the pool TWAP rather than the slot0 spot tick to remove the spot manipulation vector, and treat the half width as a required per pair configuration parameter with no silent universal default, since no single value fits all pairs.

Status

Acknowledged, no fix implemented. It is presumed that for general V3 token pairs, the fallback tick range of 5% is sufficient to present this pairs.

For current LST deployments, the original 1% range is utilized, so this is latent to future general pairs.

[L07] Dust removeLiquidity burns shares but can return zero principal

Contract V3DexAdapter.sol

Severity Low

Description

In removeLiquidity, the amount of liquidity to withdraw is computed as $\text{uint128}(\text{totalLiq} * \text{shares} / \text{totalShares})$. For a sufficiently small shares / totalShares ratio this division rounds down to 0. When it is 0, the branch that calls decreaseLiquidity and collectAll is skipped entirely, which also skips the minAmount slippage check, since that check is only applied inside decreaseLiquidity. The pro rata idle inventory is computed the same way and may likewise round down to 0.

Description

JavaScript

```
/// @inheritdoc IV3DexAdapter
function removeLiquidity(
    uint256 shares,
    uint256 totalShares,
    uint256 minAmount0,
    uint256 minAmount1,
    address receiver
) external onlyProvider nonReentrant returns (uint256 amount0, uint256
amount1) {
    uint128 totalLiq = _getPositionLiquidity();
    uint128 liquidityToRemove = totalShares == 0 ? 0 :
uint128((uint256(totalLiq) * shares) / totalShares);

    if (liquidityToRemove > 0) {
        V3PositionLib.decreaseLiquidity(POSITION_MANAGER, tokenId,
liquidityToRemove, minAmount0, minAmount1);
        (amount0, amount1) = V3PositionLib.collectAll(POSITION_MANAGER,
tokenId);
    }
}
```

```

    // Pro-rata idle inventory (finding C): redeem the same fraction of
    idle as of liquidity.
    if (totalShares > 0) {
        uint256 idleOut0 = (idleToken0 * shares) / totalShares;
        uint256 idleOut1 = (idleToken1 * shares) / totalShares;
        if (idleOut0 > 0) {
            idleToken0 -= idleOut0;
            amount0 += idleOut0;
        }
        if (idleOut1 > 0) {
            idleToken1 -= idleOut1;
            amount1 += idleOut1;
        }
    }

    if (amount0 > 0) _sendToken(TOKEN0, amount0, payable(receiver));
    if (amount1 > 0) _sendToken(TOKEN1, amount1, payable(receiver));

    emit LiquidityRemoved(shares, totalShares, amount0, amount1,
        receiver);
}

```

Impact

The provider burns the caller's shares before invoking `removeLiquidity`, so when both the liquidity portion and the idle portion round to 0, a dust redemption destroys the caller's shares and returns (0, 0) with no revert. The caller receives nothing for the burned shares, and the `minAmount` guard they supplied does not trigger because the code path that enforces it never runs.

All divisions here round down, so the redeemer always receives at most their fair share and the remainder stays in the contract. There is no gain vector: repeating small redemptions only forfeits more value to the remaining holders rather than extracting from the reserves, which is why the severity is Low. The concrete impact is a real value loss for users who withdraw very small amounts, and a related condition where a holder whose total balance is small enough that every portion rounds to 0 cannot redeem any value at all.

Recommendation	Consider requiring that the total computed output, meaning liquidity removed plus idle removed, is greater than 0 and revert otherwise,
Status	Fixed, now includes a check that reverts if <code>amount0</code> or <code>amount1</code> is zero when shares passed into <code>performRedemptions/withdrawals</code> is more than zero.

[L08] `previewAddLiquidity` and `previewRemoveLiquidity` do not use `fairSqrtPriceX96` for liquidity previews and does not simulate fee compounding

Contract	V3DexAdapter.sol
----------	------------------

Severity	Low
----------	-----

Description

The V3DexAdapter.previewAddLiquidity() and previewRemoveLiquidity() functions calculate expected liquidity outcomes using a price source that differs from the one applied during actual market actions in the V3Provider. When the V3Provider adds or removes liquidity, it relies on the fairSqrtPriceX96 to determine the position amounts.. The preview functions, however, do not use this same fairSqrtPriceX96 value, so the previewed results can diverge from what the real operations produce.

Description

JavaScript

```

/// @inheritdoc IV3DexAdapter
function previewAddLiquidity(
    uint256 amount0Desired,
    uint256 amount1Desired
) external view returns (uint128 liquidity, uint256 amount0, uint256
amount1) {
    uint160 sqrtPriceX96 = spotSqrtPriceX96();
    uint160 sqrtLower = TickMath.getSqrtRatioAtTick(tickLower);
    uint160 sqrtUpper = TickMath.getSqrtRatioAtTick(tickUpper);
    liquidity = LiquidityAmounts.getLiquidityForAmounts(
        sqrtPriceX96,
        sqrtLower,
        sqrtUpper,
        amount0Desired,
        amount1Desired
    );
    (amount0, amount1) =
    LiquidityAmounts.getAmountsForLiquidity(sqrtPriceX96, sqrtLower,
sqrtUpper, liquidity);
}

/// @inheritdoc IV3DexAdapter
function previewRemoveLiquidity(
    uint256 shares,
    uint256 totalShares
) external view returns (uint256 amount0, uint256 amount1) {
    if (totalShares == 0 || shares == 0) return (0, 0);
    uint128 liquidityToRemove =
uint128((uint256(_getPositionLiquidity()) * shares) / totalShares);
    (amount0, amount1) = LiquidityAmounts.getAmountsForLiquidity(
        spotSqrtPriceX96(),
        TickMath.getSqrtRatioAtTick(tickLower),
        TickMath.getSqrtRatioAtTick(tickUpper),
        liquidityToRemove
    );
    amount0 += (idleToken0 * shares) / totalShares;
    amount1 += (idleToken1 * shares) / totalShares;
}

```

Additionally, the state-changing paths (deposit, withdraw, redeemShares) all run collectAndCompound() first, which collects fees and folds them (plus idle) into the position's liquidity before the operation proceeds. The preview/view functions do not replicate that, therefore, the preview under-estimates the real redemption by the redeemer's pro-rata share of accrued fees.

Impact

This inconsistency means callers relying on the preview functions to estimate token amounts, minimum outputs, or slippage bounds may receive figures that do not match the executed transactions, leading to inaccurate expectations and potential mismatches between simulated and actual results..

Recommendation

Update previewAddLiquidity() and previewRemoveLiquidity() to compute liquidity additions and removals using the same fairSqrtPriceX96 that the V3Provider applies during its market actions. Aligning the preview pricing source with the execution pricing source ensures the previewed amounts accurately reflect the outcomes of the corresponding on chain operations.

Additionally, consider documenting the documentation to state the preview is a lower bound that excludes fees compounded at redemption, so integrators size minAmount with that understanding rather than treating it as exact

Status

Fixed, the spot price is still utilized for preview functions, however, sufficient documentation is included within the V3DexAdapter and IV3DexAdapter interface to indicate that spot price is utilized to compute preview.

It is although still recommended that the fairSqrtPriceX96 view function is utilized to perform accurate preview of liquidity addition and removals.

[L09] Rebalance relies on a spot dependent aggregate minLiquidity that the BOT alone sets, giving weak value protection

Contract V3DexAdapter.sol

Severity Low

Description

Description

The V3DexAdapter.rebalance function uses an aggregate minLiquidity check as its mint stage slippage guard, and that guard is a weak, spot dependent proxy for value protection. Because the BOT role alone decides every slippage parameter off chain, the rebalance's protection against value loss and against griefing is fragile.

rebalance is BOT gated at the provider, since WstETHV3Provider.rebalance and its wbETH and slisBNB equivalents are onlyRole(BOT) and forward to the adapter's

onlyProvider rebalance. A single BOT keeper therefore supplies minAmount0, minAmount1, minLiquidity, deadline, and swapData. Three separate guards exist.

- The removal is guarded by decreaseLiquidity with minAmount0 and minAmount1.
- The inventory swap is bounded by the backend's amountOutMin encoded inside swapData.
- The mint is called with amount0Min = 0 and amount1Min = 0, so the only guard on the mint is the final check that reverts when mintedLiquidity < minLiquidity.

JavaScript

```
/// @inheritdoc IV3DexAdapter
function rebalance(
  uint256 minAmount0,
  uint256 minAmount1,
  uint256 minLiquidity,
  uint256 deadline,
  bytes calldata swapData
) external onlyProvider nonReentrant {
  if (block.timestamp > deadline) revert DeadlineExpired();

  // Rate-implied pairs recenter around the LST↔native rate; pure-TWAP
  pairs (rate == 0) recenter
  // around the spot tick and skip the rate-drift guard / inventory
  conversion.
  uint256 centerRate = _lstNativeRate();
  bool rateImplied = centerRate != 0;
  if (rateImplied) _requireCenterRateDeviation(centerRate);

  (int24 newTickLower, int24 newTickUpper) =
    _initialTickRange(centerRate);
  int24 oldTickLower = tickLower;
  int24 oldTickUpper = tickUpper;

  uint256 total0;
  uint256 total1;
  if (tokenId != 0) {
    (total0, total1) = V3PositionLib.collectAll(POSITION_MANAGER,
tokenId);
  }
  total0 += idleToken0;
  total1 += idleToken1;
  idleToken0 = 0;
  idleToken1 = 0;

  if (tokenId != 0) {
    uint128 liquidity = _getPositionLiquidity();
    if (liquidity > 0) {
```

```

        V3PositionLib.decreaseLiquidity(POSITION_MANAGER, tokenId,
liquidity, minAmount0, minAmount1);
    }
    (uint256 removed0, uint256 removed1) =
V3PositionLib.collectAll(POSITION_MANAGER, tokenId);
    total0 += removed0;
    total1 += removed1;
    V3PositionLib.burn(POSITION_MANAGER, tokenId);
    tokenId = 0;
}

(total0, total1) = _convertToOptimalRatio(total0, total1,
newTickLower, newTickUpper, centerRate, swapData);

tickLower = newTickLower;
tickUpper = newTickUpper;

uint128 mintedLiquidity;
if (total0 > 0 || total1 > 0) {
    (uint256 newTokenId, uint128 liquidity, uint256 used0, uint256
used1) = V3PositionLib.mint(
        POSITION_MANAGER,
        TOKEN0,
        TOKEN1,
        FEE,
        newTickLower,
        newTickUpper,
        total0,
        total1,
        0,
        0
    );
    tokenId = newTokenId;
    mintedLiquidity = liquidity;
    idleToken0 = total0 - used0;
    idleToken1 = total1 - used1;
} else {
    idleToken0 = total0;
    idleToken1 = total1;
}

if (uint256(mintedLiquidity) < minLiquidity) revert
InsufficientLiquidityMinted();

if (rateImplied) {
    uint256 oldCenterRate = lastCenterRate;
    lastCenterRate = centerRate;
    emit LastCenterRateUpdated(oldCenterRate, centerRate);
}

```

```

        emit Rebalanced(oldTickLower, oldTickUpper, newTickLower,
newTickUpper, tokenId);
    }

```

Impact

1. The first problem is that minLiquidity is evaluated at the manipulable pool spot, not the fair price. The mint derives its liquidity from the pool's current slot0 price through getLiquidityForAmounts, so mintedLiquidity is a function of spot and the check $\text{mintedLiquidity} \geq \text{minLiquidity}$ compares against a manipulable denominator.
2. The second problem is that the check excludes idle inventory, so it is not a value conservation guard. After the mint, unused tokens are retained as $\text{idleToken0} = \text{total0} - \text{used0}$ and $\text{idleToken1} = \text{total1} - \text{used1}$, and minLiquidity bounds only the minted portion, not the total redeployed value. As a liveness and griefing consequence, an attacker who skews spot so the mint consumes a lopsided ratio pushes more value into idle and drops minted liquidity below minLiquidity, reverting the rebalance.

This is amplified by the BOT role alone computing and supplying all three slippage parameters off chain, with no on chain derivation from the fair price, a single wrong or too loose parameter, especially minLiquidity = 0 or a loose amountOutMin, silently removes the corresponding chain floor or manipulation-resistant cross check tying the allowed value loss to the fair price, so correctness depends entirely on the BOT computing every parameter correctly against a moving spot.

The severity is limited because the swap venue is whitelisted, the removal and swap guards exist, and the position is later valued at the fair price the rebalance spot. This is a robustness and griefing concern rather than a direct theft path.

Snapshot the value of the position plus idle through `positionAmountsAt(fairSqrtPriceX96())` before the rebalance and again after, then require that the post value is $\geq$ the pre value scaled by $(\text{BPS} - \text{tol}) / \text{BPS}$ for a bounded tolerance.

Recommendation

Because fairSqrtPriceX96 is rate anchored, and TWAP clamped for wstETH and wbETH, this value is not manipulable by a spot sandwich, it captures the whole operation of removal, swap, and mint in one guard, and it counts idle so it cannot be grieved by forcing value into idle.

This also reduces reliance on the BOT setting three separate parameters correctly against a moving spot. Keep minLiquidity as an optional effectiveness floor if desired,

but evaluate the mint bounds against the fair price rather than spot, and consider passing non zero amount0Min and amount1Min to the mint.

Fixed, now introduces three safeguards per rebalance call

Status

- A per rebalance maxRebalanceLossBp NAV deviation and absolute daily loss accumulator check via dailyLossAccum
- A per swap deviation maxSwapLossBp check for LST pairs via _requireSwapLossWithinCap when performing a swap to achieve optimal liquidity ratio
- The existing _requireCenterRateDeviation that enforces rebalancing when current LST rates have sufficiently deviated
- A price deviation check via _priceWithinDeviation post mint to enforce the spot must be within maxSpotDeviationBps of it, so a price that was moved off the BOT's expectation during swaps via _convertToOptimalRatio.

[L10] Stale tick range after a full exit can temporarily block re-deposits or allow deposit using stale tick ranges

Contract V3DexAdapter.sol

Severity Low

Description

In V3DexAdapter, the tickLower and tickUpper values remain frozen at whatever they were last set to after the position is fully exited. The adapter does not clear or refresh them on a full withdrawal, so they persist until the next rebalance. If the exchange rate or pool price drifts materially while the adapter sits drained and idle, the first re-deposit calls increaseLiquidity against that stale range.

Description

JavaScript

```
/// @inheritdoc IV3DexAdapter
function previewAddLiquidity(
    uint256 amount0Desired,
    uint256 amount1Desired
) external view returns (uint128 liquidity, uint256 amount0, uint256
amount1) {
    uint160 sqrtPriceX96 = spotSqrtPriceX96();
    uint160 sqrtLower = TickMath.getSqrtRatioAtTick(tickLower);
    uint160 sqrtUpper = TickMath.getSqrtRatioAtTick(tickUpper);
    liquidity = LiquidityAmounts.getLiquidityForAmounts(
        sqrtPriceX96,
        sqrtLower,
```



```

        sqrtUpper,
        amount0Desired,
        amount1Desired
    );
    (amount0, amount1) =
    LiquidityAmounts.getAmountsForLiquidity(sqrtPriceX96, sqrtLower,
    sqrtUpper, liquidity);
}

/// @inheritdoc IV3DexAdapter
function previewRemoveLiquidity(
    uint256 shares,
    uint256 totalShares
) external view returns (uint256 amount0, uint256 amount1) {
    if (totalShares == 0 || shares == 0) return (0, 0);
    uint128 liquidityToRemove =
    uint128((uint256(_getPositionLiquidity()) * shares) / totalShares);
    (amount0, amount1) = LiquidityAmounts.getAmountsForLiquidity(
        spotSqrtPriceX96(),
        TickMath.getSqrtRatioAtTick(tickLower),
        TickMath.getSqrtRatioAtTick(tickUpper),
        liquidityToRemove
    );
    amount0 += (idleToken0 * shares) / totalShares;
    amount1 += (idleToken1 * shares) / totalShares;
}

```

Impact

When a spot has moved outside the frozen range, the resulting mint is forced single-sided, or it reverts outright because the pool's mint path requires amount > 0. In that state, deposit can revert for every user until a BOT rebalance refreshes the range by burning the stale NFT and recentering on the current rate. The impact is a temporary liveness gap specific to the scenario where the adapter is drained and then left idle long enough for the price to drift, during which new deposits cannot succeed.

The condition is recoverable. A rebalance on an empty adapter recenters on the current rate and restores normal deposits. For rate-implied pairs, recovery depends on the rate deviation guard allowing the rebalance, which it does once the drift reaches its configured threshold.

An additional side effect is that rebalancing uses all of the idleToken0 and idleToken1 to perform liquidity addition to the updated tick range from the centerRate. This means that if a tick range is stale, a user can actually front-run to perform a small deposit/withdraw to trigger collectAndCompound that compounds the fees into the stale tick range instead of the updated fee range.

Recommendation	Ensure that routine rebalancing and refreshing of the NPM position is performed to ensure the tick range is never stale for both rate-implied and regular pairs.
Status	Acknowledged, no fix implemented, it is presumed that frequent rebalancing will be performed by the BOT role. Additionally, given ListaDao will likely seed an initial value of tokens, it is presumed that an empty V3Provider market is unlikely.

[I11] Compounded event reports attempted amounts instead of the amounts actually added

Contract	V3DexAdapter.sol
Severity	Informational
Description	Description The V3DexAdapter._collectAndCompound() function emits the Compounded event using toCompound0 and toCompound1, which are the amounts it attempts to add as liquidity, rather than used0 and used1, which are the amounts the position actually consumed. Because the leftover of each token that is not used is written back as idle inventory, the emitted values overstate how much was truly compounded. <pre> JavaScript function _collectAndCompound() internal { if (tokenId == 0) return; (uint256 fees0, uint256 fees1) = V3PositionLib.collectAll(POSITION_MANAGER, tokenId); uint256 toCompound0 = fees0 + idleToken0; uint256 toCompound1 = fees1 + idleToken1; if (toCompound0 == 0 && toCompound1 == 0) return; (uint128 liquidityAdded, uint256 used0, uint256 used1) = V3PositionLib.increaseLiquidity(POSITION_MANAGER, TOKEN0, TOKEN1, tokenId, toCompound0, toCompound1, 0, 0); idleToken0 = toCompound0 - used0; idleToken1 = toCompound1 - used1; emit Compounded(toCompound0, toCompound1, liquidityAdded); } </pre>

	Impact Any off chain system that tracks compounding activity from this event will record inflated figures and will not see the split between what became active liquidity and what remained idle. This is a reporting inaccuracy with no effect on funds or contract state.
Recommendation	Emit the Compounded event with used0 and used1 so it reflects the amounts actually added to the position. If visibility into the attempted totals is also desired, extend the event to include both the attempted amounts (toCompound0 and toCompound1) and the used amounts (used0 and used1) so consumers can see the used portion alongside the amount left as idle.
Status	Fixed as recommended, now emits used0 and used1 instead of toCompound0 and toCompound1 to reflect the actual underlying tokens utilized for compounding fees.

[I12] Redundant collectAll before decreaseLiquidity in rebalance wastes an NPM call

Contract	V3DexAdapter.sol
Severity	Informational
Description	Description In V3DexAdapter.rebalance, the position's tokens are collected twice. The first collectAll runs before decreaseLiquidity to sweep accrued fees, and a second collectAll using type(uint128).max runs after decreaseLiquidity to sweep the freed principal. The first collection is redundant. When decreaseLiquidity runs, it credits both the accrued fees and the freed principal into the position's tokensOwed, so the later collectAll with type(uint128).max already sweeps the fees together with the principal. <pre> JavaScript uint256 total0; uint256 total1; if (tokenId != 0) { (total0, total1) = V3PositionLib.collectAll(POSITION_MANAGER, tokenId); } total0 += idleToken0; total1 += idleToken1; idleToken0 = 0; idleToken1 = 0; if (tokenId != 0) { uint128 liquidity = _getPositionLiquidity(); if (liquidity > 0) { V3PositionLib.decreaseLiquidity(POSITION_MANAGER, tokenId, liquidity, minAmount0, minAmount1); } (uint256 removed0, uint256 removed1) = V3PositionLib.collectAll(POSITION_MANAGER, tokenId); </pre>

```

total0 += removed0;
total1 += removed1;
V3PositionLib.burn(POSITION_MANAGER, tokenId);
tokenId = 0;
}

```

Impact

Both sequences end with the same total token amounts, so removing the first collectAll produces no behavioral change while eliminating one unnecessary NonfungiblePositionManager call and its associated gas cost.

Recommendation

Consider removing the first collectAll that runs before decreaseLiquidity in rebalance, and rely on the single collectAll with type(uint128).max performed after decreaseLiquidity to sweep both the accrued fees and the freed principal.

Status

Acknowledged, no fix implemented. It is encouraged to implement this fix to reduce gas consumption, although it does not affect core code logic.

[I13] Fallback and extreme-rate tick ranges can exceed MIN_TICK / MAX_TICK and revert the mint

Contract V3DexAdapter.sol

Severity Informational

Description

In the pure TWAP fallback path (`centerRate == 0`), `_initialTickRange()` sets the lower bound to `currentTick - FALLBACK_HALF_RANGE_TICKS` (500) and the upper bound to `currentTick + FALLBACK_HALF_RANGE_TICKS`, then floors and ceils to `tickSpacing`. `currentTick` is read from the pool `slot0` and can be anywhere in `[MIN_TICK, MAX_TICK]` = `[-887272, 887272]`.

Description

```

JavaScript
function _initialTickRange(
    uint256 centerRate
) internal view returns (int24 initialTickLower, int24
initialTickUpper) {
    int24 tickSpacing = IListV3Pool(P00L).tickSpacing();

    if (centerRate != 0) {
        (initialTickLower, initialTickUpper) =
_tickRangeForRate(centerRate, tickSpacing);
    } else {
        (, int24 currentTick) = IV3PoolMinimal(P00L).slot0();
        initialTickLower = _floorTick(currentTick -
FALLBACK_HALF_RANGE_TICKS, tickSpacing);
    }
}

```

```

        initialTickUpper = _ceilTick(currentTick +
FALLBACK_HALF_RANGE_TICKS, tickSpacing);
    }

    // _floorTick / _ceilTick can push an aligned tick just past the
usable range (and TickMath /
// pool.mint would then revert). Clamp both bounds back into
[MIN_TICK, MAX_TICK].
    int24 minUsable = _ceilTick(TickMath.MIN_TICK, tickSpacing);
    int24 maxUsable = _floorTick(TickMath.MAX_TICK, tickSpacing);
    if (initialTickLower < minUsable) initialTickLower = minUsable;
    if (initialTickUpper > maxUsable) initialTickUpper = maxUsable;

    if (initialTickLower >= initialTickUpper) {
        initialTickLower = maxUsable - tickSpacing;
        initialTickUpper = maxUsable;
    }
}

```

When the pool's current tick is within roughly 500 ticks (plus the ceil rounding) of MAX_TICK or MIN_TICK, the computed upper (or lower) tick exceeds the valid tick bounds, and the subsequent pool mint through TickMath.getSqrtRatioAtTick reverts, so rebalance and the first liquidity add cannot proceed for such an extreme price pool. The same can occur on the rate path if an extreme center rate maps tickAt(1.01 * centerRate) beyond MAX_TICK.

Impact

This only affects pools operating near the tick extremes (pathological price ratios or large decimal differences) and is not reachable for the in scope LST / native pairs, whose prices sit near the middle of the tick range. It remains a latent revert for a future general pair.

Recommendation

Consider never supporting V3 token pairs of extreme price ranges

Status

Fixed, the _initialTickRange now prevents _floor/_ceilTick from pushing a bound past the usable range (which TickMath/pool.mint would reject). By computing the minUsable and maxUsable tick range.

[I14] The initialTickLower >= initialTickUpper guard is unreachable with the current constants

Contract V3DexAdapter.sol

Severity Informational

Description Description

The guard `if (initialTickLower >= initialTickUpper)` `initialTickUpper = initialTickLower + tickSpacing`; in `_initialTickRange()` cannot execute with the current constants. Since `_floorTick(a) <= a` and `_ceilTick(b) >= b`, whenever the raw lower bound `a` is strictly less than the raw upper bound `b`, it follows that `_floorTick(a) < _ceilTick(b)` strictly. Both code paths guarantee `a < b` by a wide margin, whereby the rate path separates the bounds by about 202 ticks (the 2% gap between $0.99 * \text{rate}$ and $1.01 * \text{rate}$, far more than one tick), and the fallback path separates them by 1000 ticks.

JavaScript

```
/// @dev Tick range for the position. Rate-implied (centerRate != 0):
±INITIAL_RANGE_BPS around the
/// rate-derived price. Pure-TWAP (centerRate == 0):
±FALLBACK_HALF_RANGE_TICKS around spot.
function _initialTickRange(
    uint256 centerRate
) internal view returns (int24 initialTickLower, int24
initialTickUpper) {
    int24 tickSpacing = IListV3Pool(P00L).tickSpacing();

    if (centerRate != 0) {
        (initialTickLower, initialTickUpper) =
_tickRangeForRate(centerRate, tickSpacing);
    } else {
        (, int24 currentTick) = IV3PoolMinimal(P00L).slot0();
        initialTickLower = _floorTick(currentTick -
FALLBACK_HALF_RANGE_TICKS, tickSpacing);
        initialTickUpper = _ceilTick(currentTick +
FALLBACK_HALF_RANGE_TICKS, tickSpacing);
    }

    if (initialTickLower >= initialTickUpper) {
        initialTickUpper = initialTickLower + tickSpacing;
    }
}
```

Impact

The guard could only fire if the raw width were zero (`rawLower == rawUpper`), which requires a width constant of zero (`INITIAL_RANGE_BPS == 0` or `FALLBACK_HALF_RANGE_TICKS == 0`). Both are non zero compile time constants (100 and 500), so the condition is never met. The guard reads as a defensive leftover from a design where the half width was parameterizable.

Recommendation	Consider removing this guard, if not, we can keep the guard as defensive protection, but document that it is unreachable under the current constants.
Status	Acknowledged, no fix implemented, logic kept as a safeguard.

[I15] Redundant else-branch assignments in rebalance() when the adapter is empty

Contract V3DexAdapter.sol

Severity Informational

Description

In V3DexAdapter.rebalance(), after the inventory is collected and converted, the mint block is guarded by `if (total0 > 0 || total1 > 0)`. The else branch assigns `idleToken0 = total0` and `idleToken1 = total1`, but this branch only executes when `total0 == 0 && total1 == 0`. Earlier in the same function the idle balances were already reset (`idleToken0 = 0`; `idleToken1 = 0`) before the position was decreased and burned, so the else branch merely re-assigns zero to variables that are already zero.

Description

JavaScript

```
total0 += idleToken0;
total1 += idleToken1;
idleToken0 = 0;
idleToken1 = 0;

if (tokenId != 0) {
    uint128 liquidity = _getPositionLiquidity();
    if (liquidity > 0) {
        V3PositionLib.decreaseLiquidity(POSITION_MANAGER, tokenId,
liquidity, minAmount0, minAmount1);
    }
    (uint256 removed0, uint256 removed1) =
V3PositionLib.collectAll(POSITION_MANAGER, tokenId);
    total0 += removed0;
    total1 += removed1;
    V3PositionLib.burn(POSITION_MANAGER, tokenId);
    tokenId = 0;
}

(total0, total1) = _convertToOptimalRatio(total0, total1,
newTickLower, newTickUpper, centerRate, swapData);

tickLower = newTickLower;
tickUpper = newTickUpper;

uint128 mintedLiquidity;
if (total0 > 0 || total1 > 0) {
    (uint256 newTokenId, uint128 liquidity, uint256 used0, uint256
used1) = V3PositionLib.mint(
        POSITION_MANAGER,
        TOKEN0,
        TOKEN1,
        FEE,
        newTickLower,
        newTickUpper,
        total0,
```

```

        total1,
        0,
        0
    );
    tokenId = newTokenId;
    mintedLiquidity = liquidity;
    idleToken0 = total0 - used0;
    idleToken1 = total1 - used1;
} else {
    idleToken0 = total0;
    idleToken1 = total1;
}

```

Impact

The branch is reached only when there is nothing to redeploy. This happens for a fresh adapter that has never received a deposit (`tokenId == 0`, and `idle == 0` because the invariant holds that `idle > 0` implies `tokenId != 0`), or for a fully-redeemed adapter (`tokenId != 0` but `liquidity == 0` and `idle == 0`, whose stale NFT is then burned). The only observable consequence is a `Rebalanced` event emitted with `tokenId == 0`, which off-chain indexers should be prepared to handle.

Recommendation	Consider removing the redundant else assignments, since the idle balances are already zero at that point, or replace them with a clarifying comment.
Status	Acknowledged, no fix implemented, kept for future safeguards.

[I16] Unused `targetTickLower`, `targetTickUpper`, and `rate` parameters in `_convertToOptimalRatio`

Contract V3DexAdapter.sol

Severity Informational

Description

`V3DexAdapter._convertToOptimalRatio` is a virtual extension hook called once by `rebalance`, which passes it the full conversion context: the current inventory (`total0`, `total1`), the target range (`targetTickLower`, `targetTickUpper`), the exchange rate (`rate`), and the backend-built `swapData`. The base implementation only consumes `total0`, `total1`, and `swapData`, executing the backend-built swap through `SwapInventoryLib` and letting the `minLiquidity` check in `rebalance` bound the result. It never reads `targetTickLower`, `targetTickUpper`, or `rate`, and those three parameters are commented out in the signature to silence the Solidity unused-parameter warning while keeping their types so any override retains a stable interface.

JavaScript

```
    /// @dev DEX-agnostic, backend-built rebalance inventory conversion,
    shared by all rate-implied pairs
    ///      (slisBNB/WBNB, wstETH/WETH, wbETH/WETH). `swapData` (when
    non-empty) ABI-encodes
    ///      (address swapPair, bool sellToken0, uint256 amountIn, uint256
    amountOutMin, bool nativeIn,
    ///      bytes innerSwapData): the adapter requires `swapPair`
    whitelisted and forwards `innerSwapData`
    ///      via a low-level call, bounding the swap by the backend's
    `amountOutMin` (see {SwapInventoryLib}).
    ///      `nativeIn`  $\Rightarrow$  a native-input venue (the wrapped-native leg is
    unwrapped + sent as msg.value, e.g.
    ///      StakeManager.deposit). Empty swapData  $\Rightarrow$  recenter without
    converting (also the TWAP-pair default).
    function _convertToOptimalRatio(
        uint256 total0,
        uint256 total1,
        int24 /* targetTickLower */,
        int24 /* targetTickUpper */,
        uint256 /* rate */,
        bytes calldata swapData
    ) internal virtual returns (uint256, uint256) {
        if (swapData.length == 0) return (total0, total1);
        (address swapPair, bool sellToken0, uint256 amountIn, uint256
        amountOutMin, bool nativeIn, bytes memory inner) = abi
            .decode(swapData, (address, bool, uint256, uint256, bool, bytes));
        if (!swapPairWhitelist[swapPair]) revert NotWhitelistedPair();
        return
            SwapInventoryLib.swap(
                swapPair,
                TOKEN0,
                TOKEN1,
                sellToken0,
                amountIn,
                amountOutMin,
                inner,
                total0,
                total1,
                WRAPPED_NATIVE,
                nativeIn
            );
    }
}
```

Impact

No subclass currently overrides `_convertToOptimalRatio`. The three deployed adapters (SlisBNBV3DexAdapter, WstETHV3DexAdapter, WbETHV3DexAdapter) override only `_lstNativeRate` and, for the two ETH pairs, `fairSqrtPriceX96`. So across all deployments the three parameters are dead inputs.

	The parameters likely exist so a future override can compute or verify the optimal token ratio on-chain rather than trusting the off-chain BOT. <code>targetTickLower</code> and <code>targetTickUpper</code> combined with the current pool price give the exact token0 to token1 ratio that maximizes deployed liquidity, and <code>rate</code> gives the manipulation-resistant fair price against which a swap result could be validated. The default path instead delegates the ratio computation entirely to the BOT and bounds the outcome only with <code>minLiquidity</code> and the BOT-supplied <code>amountOutMin</code>. As such the unused parameters signal that on-chain optimality or verification was anticipated in the interface but not implemented.
Recommendation	Consider either keeping the parameters and adding a short comment on <code>_convertToOptimalRatio</code> stating that the base implementation intentionally delegates ratio computation to the off-chain BOT and that <code>targetTickLower</code>, <code>targetTickUpper</code>, and <code>rate</code> are provided only for subclass overrides that compute or verify the ratio on-chain, or remove the three parameters until an override actually needs them and reintroduce them at that point.
Status	Acknowledged, <code>targetTickLower</code> and <code>targetTickUpper</code> kept for future safeguards. The <code>rate</code> input is now utilized for the <code>_requireSwapLossWithinCap</code> call to enforce a loss guard for BOT driven swaps to ensure the NPM liquidity is in an optimal ratio.

[I17] `_twapTick` omits the unchecked wrap of the `OracleLibrary`, so it reverts on tickCumulative overflow

Contract	V3DexAdapter.sol
Severity	Informational
Description	Description V3DexAdapter.<code>_twapTick</code> reproduces the floor-rounded mean-tick calculation of the canonical Uniswap and PancakeSwap V3 OracleLibrary.consult, computing <code>int56 delta = tickCumulatives[1] - tickCumulatives[0]</code>, then <code>twapTick = int24(delta / int56(uint56(TWAP_PERIOD)))</code>, and decrementing when <code>delta < 0</code> and the modulo is non-zero. The reference <code>OracleLibrary.sol</code> in the <code>lista-v3</code> periphery compiles under <code>pragma solidity >=0.5.0 <0.8.0</code>, and the producer-side accumulator in the core <code>Oracle.sol</code> transform function, which does <code>last.tickCumulative + int56(tick) * delta</code>, compiles under the same pre-0.8 pragma. In Solidity before 0.8, arithmetic wraps by default, and Uniswap V3 intends the <code>int56</code> tickCumulative accumulator to overflow and wrap. Because both the producer addition and the consumer subtraction wrap by the same 2^{56} modulus, the delta is recovered correctly even when the accumulator has crossed the <code>int56</code> boundary. <pre> JavaScript function _twapTick() internal view returns (int24 twapTick) { uint32[] memory secondsAgos = new uint32[](2); </pre>

```

secondsAgos[0] = TWAP_PERIOD;
secondsAgos[1] = 0;
(int56[] memory tickCumulatives, ) =
IListaV3Pool(P00L).observe(secondsAgos);
int56 delta = tickCumulatives[1] - tickCumulatives[0];
twapTick = int24(delta / int56(uint56(TWAP_PERIOD)));
if (delta < 0 && (delta % int56(uint56(TWAP_PERIOD)) != 0))
twapTick--;
}

```

Impact

V3DexAdapter compiles under pragma solidity 0.8.34, where arithmetic is checked, and the subtraction is not placed in an unchecked block. As a result the wrap cancellation is lost. If the pool's tickCumulative crosses the int56 boundary within a single TWAP observation window, the mathematical value of tickCumulatives[1] - tickCumulatives[0] has magnitude near 2^{56} , which is outside the int56 range, so the checked subtraction reverts instead of returning the correct small delta that the wrapping reference would produce.

The trigger is unreachable in any practical horizon. int56 wraps at about $3.6e16$. tickCumulative grows by tick multiplied by elapsed seconds, so at the theoretical MAX_TICK of 887272 the boundary is reached in roughly 1287 years, and for the in-scope correlated pairs, where the tick is only a few hundred to about 1823, it takes on the order of 570000 years of continuous operation from pool genesis. The revert also only is invoked if the boundary is crossed inside one 30-minute window, meaning the accumulator must already sit at the boundary.

If the condition ever occurred, _twapTick would revert, which would revert fairSqrtPriceX96 for the wstETH and wbETH adapters whenever maxTwapDeviationBps > 0, which then reverts V3ProviderOracle.peek and blocks collateral pricing, borrows, and liquidations for those markets. The slisBNB adapter is unaffected because its pure-rate fairSqrtPriceX96 never calls _twapTick.

Recommendation	Consider wrapping the delta computation in an unchecked block to match the canonical OracleLibrary.consult semantics and restore the intended wrap cancellation, if not document appropriately and never utilize any pair that could allow such an overflow to occur.
Status	Fixed, now utilizes the unchecked semantics similar to the OracleLibrary.consult.

[I18] __V3DexAdapter_init does not enforce tick spacing alignment or tick bounds on the initial range

Contract V3DexAdapter.sol

Severity **Informational**

Description

V3DexAdapter.__V3DexAdapter_init() stores the supplied _tickLower and _tickUpper after checking only that _admin and _manager are non-zero and that _tickLower < _tickUpper. It does not verify that the two ticks are aligned to the pool's tickSpacing, nor that they fall within MIN_TICK and MAX_TICK. The base contract treats "the initial range is aligned and in bounds" as an invariant it delegates to every inheriting contract rather than something it enforces.

JavaScript

```
function __V3DexAdapter_init(
  address _admin,
  address _manager,
  int24 _tickLower,
  int24 _tickUpper
) internal onlyInitializing {
  if (_admin == address(0) || _manager == address(0)) revert
ZeroAddress();
  if (_tickLower >= _tickUpper) revert InvalidTickRange();

  __AccessControl_init();
  __ReentrancyGuard_init();

  _grantRole(DEFAULT_ADMIN_ROLE, _admin);
  _grantRole(MANAGER, _manager);

  tickLower = _tickLower;
  tickUpper = _tickUpper;
}
```

Description

Impact

In the current deployment this is safe, because all three subclasses, SlisBNBV3DexAdapter, WstETHV3DexAdapter, and WbETHV3DexAdapter, derive the range from _initialTickRange(initialCenterRate), which floors and ceils to tickSpacing and bumps the degenerate equal case, so the stored range is always aligned and ordered. The concern is latent. A future subclass, or a refactor that passes a raw range, would pass initialization cleanly but leave the adapter unusable, since the first addLiquidity call routes into the NonfungiblePositionManager mint, which requires ticks aligned to tickSpacing and reverts otherwise. Because initialize runs only once and there is no dedicated setter for the tick range, recovery would depend on a BOT rebalance re-deriving an aligned range. There is no loss of funds and no live exploit, so this is informational.

Additionally, __V3DexAdapter_init initializes neither lastCenterRate nor centerRateThresholdBps, where each subclass sets both after the base initializer returns. All three do so correctly today, but if a subclass set one and omitted the other, _requireCenterRateDeviation would short circuit on thresholdBps == 0 or

	previousCenterRate == 0 and the first rebalance's rate drift guard would be silently disabled, with no revert to signal the misconfiguration.
Recommendation	Consider having the base derive the initial range itself rather than accept it as a parameter, for example by computing <code>_initialTickRange</code> inside <code>__V3DexAdapter_init</code> from a rate or spot value the subclass provides through a hook. If the parameterized form is kept, assert in the initializer that both ticks are multiples of <code>IListaV3Pool(POOL).tickSpacing()</code> and lie within <code>MIN_TICK</code> and <code>MAX_TICK</code>. Additionally document required state setting for rate implied pairs for subclasses inheriting the <code>V3DexAdapter</code>.
Status	Acknowledged, no fix implemented, it is presumed that the deployer performing new V3 LP Collateral market deployments will only input appropriate tick ranges when performing V3 LP Collateral deployment.

[I19] Coarse tickSpacing can collapse the rate-centered range to a single spacing

Contract	V3DexAdapter.sol
Severity	Informational
Description	Description V3DexAdapter centers the managed position on a $\pm 1\%$ band around the exchange rate via <code>_tickRangeForRate</code>, flooring the lower bound and ceiling the upper bound to the pool's <code>tickSpacing</code>. The $\pm 1\%$ band spans roughly 199 ticks. If a pool has a large <code>tickSpacing</code>, for example a 1% fee tier where the spacing is 200, that 199 tick band can round to a single spacing wide range. The effect is capital inefficiency, meaning an over concentrated or lopsided position that captures fewer fees and needs to be rebalanced more often. <pre> JavaScript function _tickRangeForRate(uint256 centerRate, int24 tickSpacing) internal view returns (int24 initialTickLower, int24 initialTickUpper) { uint256 lowerRate = (centerRate * (BPS - INITIAL_RANGE_BPS)) / BPS; uint256 upperRate = (centerRate * (BPS + INITIAL_RANGE_BPS)) / BPS; initialTickLower = _floorTick(_tickAtSqrtRatio(_sqrtPriceX96FromRate(lowerRate)), tickSpacing); initialTickUpper = _ceilTick(_tickAtSqrtRatio(_sqrtPriceX96FromRate(upperRate)), tickSpacing); } </pre>

Impact

The mitigation of asserting that the center tick lies within the range would not detect this condition. The center tick is always strictly inside the range, since flooring never raises the lower bound above its input and ceiling never lowers the upper bound below its input, while the raw endpoints already sit about 99.5 ticks on either side of the center. Concretely, the lower bound is at most center minus 99.5 and the upper bound is at least center plus 99.5, so the center is always between them and the range width is always at least one spacing, even when it collapses to exactly one spacing. An assertion on center membership would therefore never trigger and would not prevent the collapse.

For the pairs that ship today, slisBNB and WBNB, wstETH and WETH, and wbETH and WETH, this cannot occur, because those correlated pairs use a low fee tier with a small tickSpacing of 1 to 10, where the $\pm 1\%$ band resolves to roughly 100 to 200 spacings. Triggering the collapse thus requires an exotic or misconfigured high tickSpacing pool.

Recommendation

Consider ensuring that we reject any pool whose tickSpacing is too coarse relative to INITIAL_RANGE_BPS, or widen INITIAL_RANGE_BPS when the spacing is coarse so the intended $\pm 1\%$ always spans several spacings.

Status

Acknowledged, it is presumed that sufficient off-chain checks is in place to prevent accepting pools with arbitrary/coarse tick is not accepted.

[I20] _sqrtPriceX96FromRate can revert for an exotic large decimal gap pair

Contract V3DexAdapter.sol

Severity Informational

Description

V3DexAdapter._sqrtPriceX96FromRate converts the exchange rate into a pool sqrt price, adjusting for the decimal difference between the two tokens. When token1 has more decimals than token0 it pre-multiplies the rate by $10^{**}(\text{DECIMALS1} - \text{DECIMALS0})$, and in the opposite case it scales the denominator by $10^{**}(\text{DECIMALS0} - \text{DECIMALS1})$. For a pair with a very large decimal gap these operations can overflow.

Description

JavaScript

```
/// @dev Convert an exchange `rate` - token1-per-token0 scaled by 1e18
(1e18  $\Rightarrow$  1 WHOLE token0 is worth
///      1 WHOLE token1; subclasses return it in these
human/whole-token terms) - into the pool
///      sqrtPriceX96, which encodes the RAW (smallest-unit) price
token1/token0. The two tokens may have
///      different decimals (the wrapped-native is 18; the paired
token need not be), so adjust by the
///      decimal difference: token1_raw/token0_raw = (rate / 1e18) *
10^(DECIMALS1 - DECIMALS0).
```

```

function _sqrtPriceX96FromRate(uint256 rate) internal view returns
(uint160) {
    uint256 priceX192; // (token1_raw / token0_raw) * 2^192
    if (DECIMALS1 >= DECIMALS0) {
        priceX192 = FullMath.mulDiv(rate * (uint256(10) ** (DECIMALS1 -
DECIMALS0)), 1 << 192, 1e18);
    } else {
        priceX192 = FullMath.mulDiv(rate, 1 << 192, 1e18 * (uint256(10) **
(DECIMALS0 - DECIMALS1)));
    }
    return uint160(Math.sqrt(priceX192));
}

```

Impact

The first operation to fail is the mulDiv result, not the pre-multiply. With a near one to one rate the intermediate value is approximately 10^{**gap} times two to the power of 192, which exceeds a uint256 once the gap is around 20, whereas the pre-multiply alone only overflows near a gap of 59.

All pairs that ship today use 18 and 18 decimals, so the gap is zero and both scale factors are one, which makes every one of these paths unreachable. In addition, this function is reached from initialize through the initial tick range computation, so an overflow prone pair would fail immediately at deployment and could not be silently shipped into production.

Recommendation	Consider ensuring that high decimal token pairs are never utilized to be integrated into the V3DexAdapter and V3Provider.
Status	Acknowledged, it is presumed that high decimal token pairs are never integrated into the V3 LP Collateral markets.

V3Liquidator

[H01] Absolute balance profitability check in onMoolahLiquidate lets flash liquidations drain liquidator balance

Contract V3Liquidator.sol

Severity **High**

Description

The V3Liquidator flash liquidation path checks profitability using an absolute balance comparison rather than the change in balance produced by the liquidation itself, similar to the PublicLiquidator, Liquidator and BrokerLiquidator contracts.

Inside onMoolahLiquidate, after the seized V3 shares are redeemed and the collateral legs are swapped into the loan token, the only guard is a check that reverts with NoProfit when `loanToken.balanceOf(address(this)) < repaidAssets`. This compares the contract's entire loan token balance against repaidAssets and never records the balance before the redemption and swaps, so it cannot separate the loan token generated by this liquidation from loan token the contract already held.

JavaScript

```
if (d.loanToken.balanceOf(address(this)) < repaidAssets) revert  
NoProfit();
```

Description

Impact

The V3Liquidator is a pre-funded contract where the pre-funded liquidate() path requires it to hold loan token, and redeemV3Shares, proceeds from sellToken, profits from earlier liquidations, and ListaDao admin top ups all leave balances in the contract, since only the MANAGER can withdraw them. As a result, the pre-existing balance B0 at the start of the callback is generally > 0. If the seized collateral redeems and swaps into less loan token than repaidAssets, meaning $B1 - B0 < \text{repaidAssets}$, the shortfall is quietly covered by the contract's own reserves B0. The check $B1 \geq \text{repaidAssets}$ still passes, and the liquidation settles at a loss without reverting. The guard should instead require that the delta $B1 - B0 \geq \text{repaidAssets}$.

This problem is made worse by the absence of per leg slippage protection. The swaps performed inside the callback go through `_swapRedeemedLeg`, which sends the swap calldata with no minimum output and no verification of the amount received. This differs from the standalone sellToken and sellBNB functions, which snapshot balances before and after and enforce that `actualOut >= amountOutMin`.

On the flash path there is therefore neither a per leg output bound nor an aggregate delta bound. A poorly routed or sandwiched swap leg leaks value that the contract

	silently absorbs from its reserves instead of reverting, while the NoProfit guard creates a false impression that the transaction is protected against loss, although this vector depends on a malicious BOT role.
Recommendation	Similar to other Liquidator contracts, at the start of onMoolahLiquidate, before the redemption and swaps, record the current balance as before using loanToken.balanceOf(address(this)). After the redemption and swaps complete, require that the final balance is >= before + repaidAssets,
Status	Fixed, now caches a before snapshot of the loanToken balance and computes out token balance with the difference after collateral token swaps to ensure the profitability check is accurate.

[M06] V3Liquidator approval pattern reverts for revert-on-zero and approve-max-incompatible tokens

Contract	V3Liquidator.sol
Severity	Medium
Description	V3Liquidator approves through solady safeApprove and, in three places, follows a nonzero approval with an explicit reset to zero. - In liquidate it runs loanToken.safeApprove(MOOLAH, type(uint256).max), then Moolah.liquidate, then loanToken.safeApprove(MOOLAH, 0). - In _sellToken and in the ERC-20 branch of _swapRedeemedLeg it runs token.safeApprove(spender, amount), then the swap, then token.safeApprove(spender, 0). The nonzero approval and the reset to zero are both assumed to succeed for every token, but two types of tokens break this.
Description	JavaScript <pre>function redeemV3Shares(address v3Provider, uint256 shares, uint256 minAmt0, uint256 minAmt1, address receiver) external nonReentrant onlyRole(BOT) returns (uint256 amount0, uint256 amount1) { require(v3Providers[v3Provider], NotWhitelisted()); (amount0, amount1) = IV3Provider(v3Provider).redeemShares(shares, minAmt0, minAmt1, receiver); }</pre>
Impact	

1. The first class reverts on a zero-value approval. The mainnet BNB ERC-20 token at rejects zero-value operations through a `_value <= 0` check, so `approve(spender, 0)` reverts. Because the reset to zero reverts, the entire enclosing call reverts. As a result, if a token of this class is a market's loan token, the pre-funded liquidate path reverts on the reset and cannot liquidate that market, and if such a token is a redeemed collateral leg in `_swapRedeemedLeg` or a whitelisted sell target in `_sellToken`, the swap reverts and the seized collateral cannot be unwound, leaving it stuck in the contract and recoverable only through `withdrawERC20`. The callback-based `flashLiquidate` path is not affected, because its `onMoolahLiquidate` approves the exact `repaidAssets` and lets Moolah's `transferFrom` consume the allowance to zero without ever calling `approve(0)`.
2. The second class reverts on approving `type(uint256).max`. Tokens that store allowances in a narrow type such as `uint96` and do not special-case the maximum value, or that cap the approved amount, revert when liquidate approves `type(uint256).max` to Moolah, which `DoSes` the pre-funded liquidation for that loan token. The popular `uint96` tokens `UNI` and `COMP` special-case `type(uint256).max` and therefore do not revert, so this affects only niche tokens, but the pattern is not robust in general.

For the current in-scope pairs this is not triggered, since the Ethereum pairs use `wstETH`, `WETH`, and `wbETH`, the BSC side uses native BNB and standard `WBNB` and `slisBNB`, and loan tokens are standard stablecoins or wrapped native, none of which exhibit these behaviors. The exposure is conditional on a token of either class later being added as a loan token or whitelisted as a sell target, which is a `MANAGER` decision.

Recommendation

Consider approving the exact `repaidAssets` in liquidate and let Moolah's `transferFrom` consume the allowance to zero, mirroring the `flashLiquidate` path, rather than approving `type(uint256).max` and then resetting to zero.

Status

Acknowledged, no fix implemented. ListaDao has indicated that tokens with such behavior will not be whitelisted into the `V3Liquidator` contract.

Note: This means that native BNB and/or any other tokens that exhibit the above mentioned behaviors cannot be integrated into the `V3Liquidator` i.e. cannot perform liquidation of BNB markets and input token swaps that utilizes BNB or any associated tokens with these behaviors.

[I21] BOT role can divert redeemed underlying assets through the receiver parameter in `redeemV3Shares`

Contract V3Liquidator.sol

Severity Informational

Description	Description The V3Liquidator.redeemV3Shares() function lets the caller choose the receiver address that gets the underlying TOKEN0 and TOKEN1 produced by redeeming V3Provider shares, and the function is callable by the BOT role. Because the receiver is any input, a BOT account can send the redeemed assets to any address rather than keeping them in the V3Liquidator contract. <pre> JavaScript function redeemV3Shares(address v3Provider, uint256 shares, uint256 minAmt0, uint256 minAmt1, address receiver) external nonReentrant onlyRole(BOT) returns (uint256 amount0, uint256 amount1) { require(v3Providers[v3Provider], NotWhitelisted()); (amount0, amount1) = IV3Provider(v3Provider).redeemShares(shares, minAmt0, minAmt1, receiver); } </pre> Impact This sidesteps the intended fund custody model. Moving assets out of the contract is otherwise restricted to the MANAGER role through withdrawERC20 and withdrawETH. By redeeming shares directly to an external address, a BOT can extract the underlying without ever going through those MANAGER only paths.
Recommendation	Consider hardcoding the receiver to the V3Liquidator contract (i.e. address(this)) to ensure that the BOT role cannot maliciously route the redeemedV3 shares away from ListaDao admins.
Status	Acknowledged, no fix implemented. It is presumed that the BOT role is trusted and will not divert underlying redeemed amounts to an arbitrary address. It is presumed that in future iterations when the LiquidationVault is integrated in to the V3Liquidator, the receiver of redeemed shares will be held within the LiquidationVault itself subjected to swap loss guards.

[I22] Missing events in withdrawERC20 and withdrawETH

Contract	V3Liquidator.sol
Severity	Informational
Description	Description The V3Liquidator.withdrawERC20() and V3Liquidator.withdrawETH() functions move funds out of the contract but do not emit any event when they run. These are privileged

withdrawal paths, so the absence of events makes it harder to track when assets leave the contract and who initiated the transfer.

JavaScript

```
function withdrawERC20(address token, uint256 amount) external
onlyRole(MANAGER) {
    token.safeTransfer(msg.sender, amount);
}

function withdrawETH(uint256 amount) external onlyRole(MANAGER) {
    msg.sender.safeTransferETH(amount);
}
```

Impact

Off chain monitoring, accounting, and incident response all rely on emitted events, and without them these fund movements are only observable by inspecting raw transaction traces. This weakens transparency and auditability of asset outflows.

Recommendation

Consider adding a dedicated event for each function that records the relevant details of the withdrawal, such as the caller, the token address for withdrawERC20, the recipient, and the amount, then emit it inside withdrawERC20() and withdrawETH() whenever a withdrawal is performed.

Status

Acknowledged, no fix implemented. It is presumed that ListaDao utilizes the underlying ERC20 token events (if implemented) to detect MANAGER only transfers.

[I23] batchSetV3Providers omits the status equality check present in its single entry counterpart

Contract

V3Liquidator.sol

Severity

Informational

Description

The V3Liquidator.batchSetV3Providers() function updates the whitelist status of multiple V3 provider addresses in a loop, but it does not verify that the new status differs from the currently stored status for each address. The single address version of this operation rejects a no op by requiring that the incoming status is different from the existing one, yet the batch version applies the value unconditionally.

Description

JavaScript

```
function setV3ProviderWhitelist(address provider, bool status) external
onlyRole(MANAGER) {
    require(provider != address(0), "zero address");
    require(v3Providers[provider] != status, WhitelistSameStatus());
    v3Providers[provider] = status;
    emit V3ProviderWhitelistChanged(provider, status);
}
```

```
function batchSetV3Providers(address[] calldata providers, bool
status) external onlyRole(MANAGER) {
    for (uint256 i = 0; i < providers.length; i++) {
        require(providers[i] != address(0), "zero address");
        v3Providers[providers[i]] = status;
        emit V3ProviderWhitelistChanged(providers[i], status);
    }
}
```

Impact

As a result, the batch function can rewrite entries with the same status they already hold and emit a whitelist change event even though nothing actually changed. This inconsistency between the two functions can produce misleading events and redundant state writes, which complicates off chain tracking that assumes an emitted change event reflects a real status transition.

Recommendation

Consider adding the same status equality check used in the single address function to batchSetV3Providers(), so that each address in the loop is only updated and its event only emitted when the requested status differs from the current stored status. Skip or revert on entries that would not change state, matching the behavior of the single entry path.

Status

Acknowledged, no fix implemented, it is presumed that the MANAGER role would perform appropriate input checks to avoid performing V3 provider whitelist setting or the same boolean status.

[I24] Missing zero address check for token in setTokenWhitelist

Contract V3Liquidator.sol

Severity Informational

Description

The setTokenWhitelist function in V3Liquidator updates the whitelist status of a token without checking that the token address is non zero. Other whitelist setters in the same contract, such as setPairWhitelist and setV3ProviderWhitelist, explicitly reject the zero address, so this function is inconsistent with the validation pattern already established elsewhere in the contract.

Description

JavaScript

```
function setTokenWhitelist(address token, bool status) external
onlyRole(MANAGER) {
    require(tokenWhitelist[token] != status, WhitelistSameStatus());
    tokenWhitelist[token] = status;
    emit TokenWhitelistChanged(token, status);
}
```

```
}
```

Impact

The function is restricted to the MANAGER role, so the impact is limited to a configuration mistake in which the zero address is added to the token whitelist. This does not create a direct security risk, but it permits an invalid entry that has no legitimate use and can mask a misconfiguration. Aligning the check across all whitelist setters keeps the access control surface consistent and easier to reason about.

Recommendation

Consider a validation check in setTokenWhitelist that reverts when the token parameter equals the zero address, matching the checks already present in setPairWhitelist and setV3ProviderWhitelist.

Status

Acknowledged, no fix implemented. It is presumed that the MANAGER role would perform appropriate input checks to avoid zero address inputs during token whitelist.

[I25] V3Liquidation event reports amount0 and amount1 as zero on the redeem path in flashLiquidate

Contract

V3Liquidator.sol

Severity

Informational

Description

The V3Liquidator.flashLiquidate function emits the V3Liquidation event with amount0 and amount1 always set to 0. The other fields are correct, since seized and repaid carry the real return values from Moolah.liquidate, but the two token amount fields never reflect actual values.

JavaScript

```
emit V3Liquidation(id, params.v3Provider, borrower, _seized, _repaid, 0, 0);
```

Description

Impact

The cause is structural. The real redeemed leg amounts are produced inside the onMoolahLiquidate callback, where redeemShares returns amount0 and amount1, and this only happens when redeemShares == true. Those values are local to the callback and are never passed back up to flashLiquidate, so the outer emit has nothing to report and passes literal zeros.

This means the zeros are only wrong on the redeem path. When redeemShares == true, the emitted amount0 and amount1 should carry the amounts actually redeemed but

	instead report 0, so off chain consumers that read this event cannot see how much of each token the liquidation produced. When redeemShares == false, no redemption occurs because the shares are held as an ERC-20, so 0, 0 is the correct value in that branch and not a defect. The impact is limited to event data quality on the redeem path, with no effect on funds or accounting.
Recommendation	Consider propagating the real redeemed amounts to the event on the redeem path while preserving zero on the non redeem path. When redeemShares == true, capture the amount0 and amount1 returned by redeemShares inside the callback and make them available to the emit,
Status	Fixed, actual amount0 and amount1 cached within the newly introduced _redeemedAmount0 and _redeemedAmount1 state variables within the onMoolahLiquidate and emitted within the V3Liquidation event.

[I26] V3Liquidator hardcoded BNB_ADDRESS and sellBNB naming

Contract	V3Liquidator.sol
Severity	Informational
Description	The V3Liquidator contract is designed to be chain agnostic and can be deployed on more than one network, yet it uses a constant named BNB_ADDRESS to represent the native coin in its token whitelists. This name assumes the native coin is BNB, which is only accurate on BSC. On other deployments such as Ethereum the native coin is ETH, so the BNB_ADDRESS label does not match the actual asset it represents. <pre> JavaScript /// @dev Virtual address used to represent the native coin in token whitelists. address public constant BNB_ADDRESS = 0xEeeeeEeeeEeEeeEeEeEeeEEEEeeeeEeeeeeeeEEeE; </pre> Similarly, the sellBNB function can be renamed to sellETH to better represent native token sales on mainnet, along with adjusting its relevant code comment of "Sell native BNB held by this contract" <pre> JavaScript /// @notice Sell native BNB held by this contract. function sellBNB(address pair, address tokenOut, uint256 amountIn, uint256 amountOutMin, bytes calldata swapData </pre>

```
) external nonReentrant onlyRole(BOT) {
```

Impact

The value itself is the standard placeholder address used for the native coin, so functionality is unaffected, but the name is misleading and can cause confusion for integrators, reviewers, and future maintainers reading the code on a non BSC deployment.

Recommendation

Consider renaming the constant and function to a chain neutral identifier such as NATIVE_TOKEN/NATIVE_ADDRESS and sellETH/sellNative so the name reflects the native coin of whichever chain the contract is deployed on rather than hardcoding BNB in the name.

Status

[Open / Fixed / Acknowledged]

[I27] Unused IERC20 import in V3Liquidator

Contract

V3Liquidator.sol

Severity

Informational

Description

Description

The V3Liquidator contract imports the IERC20 interface but never references it anywhere in the code. Token interactions are handled through SafeTransferLib on the address type, so the IERC20 import serves no purpose.

JavaScript

```
import { ReentrancyGuardUpgradeable } from
"@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol";
```

Impact

This is a code cleanliness issue with no impact on functionality or security. Leaving unused imports in place adds needless clutter, can slightly increase compilation overhead, and may mislead readers into thinking the interface is used.

Recommendation

Consider removing the unused IERC20 import from V3Liquidator.

Status

Not fixed, the unused IERC20 interface is not removed from the V3Liquidator contract.

[I28] V3Liquidator integration considerations

Contract	V3Liquidator
Severity	Informational
Description	The following are an non-exhaustive list of considerations for future integration of the LiquidationVault with the V3Liquidator contract, given currently, every BOT swap path bounds the output only by a BOT-supplied amountOutMin (which can be 0) and never forces the swap output back to the contract, so a compromised or misused BOT can route the swap through a legitimate whitelisted router with to = attacker and drain value, defeating the deliberate design choice that the BOT is denied any withdrawal function. 1. First, redeemV3Shares is onlyRole(BOT) and forwards a caller supplied receiver straight into IV3Provider.redeemShares. A compromised BOT can redeem V3 shares held by the liquidator, which are seized collateral, and send the underlying token0 and token1 to any address it chooses, bypassing the MANAGER only withdraw. Ensure that these funds after redemption are reflowed back to the LiquidationVault 2. Similar Liquidator and BrokerLiquidator as mentioned in [M03], the flash path with redeemShares == true swaps token0 and token1 into loanToken using BOT supplied swap calldata and checks only that <code>d.loanToken.balanceOf(address(this)) < repaidAssets</code> reverts. There is no oracle loss guard and no daily cap, so a BOT can route the swap through a venue it controls and leak the liquidation bonus, the same issue identified for the other liquidators. 3. The pooled inventory model does not fit V3 collateral. The collateral of a V3 market is the V3Provider share token, which is not tradeable on a whitelisted pair and can only be redeemed through the provider, which the vault has no integration for. If the basic liquidate path, which holds shares, reflowed them, the shares would be permanently stranded in the vault. So the V3Liquidator must always redeem shares into token0 and token1 inside the liquidator and reflow the underlying, never the shares. 4. The basic liquidate function passes empty callback data, so Moolah never invokes onMoolahLiquidate. The contract must already hold the loanToken, which it max approves to Moolah with <code>type(uint256).max</code> and then clears to 0. There is therefore no callback in which to pull the exact shortfall from the vault. Vault funding of this path would require pulling before the Moolah call, but <code>repaidAssets</code> is not known until Moolah computes it, so the contract would have to over pull and reflow the remainder, or route vault funded liquidations through a callback path instead.

5. the V3Liquidator needs the
 - a. fundSource + setFundSource (with the same "vault must already list me" guard)
 - b. withdraw* relaxed from onlyRole(MANAGER) to MANAGER || msg.sender == fundSource so the vault's collect* can pull
 - c. an else if (fundSource != address(0)) pull branch in onMoolahLiquidate (for the non-redeem flash path)
 - d. _reflow of loanToken, token0, token1, and native after each liquidation/redeem (but not the V3 shares)
 - e. set _setRoleAdmin(BOT, MANAGER) in initialize (no separate setBotRoleAdmin migration needed if it deploys fresh).
6. There are native currency considerations. The redeem path unwraps the wrapped native leg to the native coin and, when the loanToken equals the wrapped native, wraps the entire contract native balance back, which greedily captures any unrelated native the contract holds.

Recommendation	See above description, a separate audit should be scheduled once LiquidationVault support is applied for the V3Liquidator
-----------------------	---

Status	Acknowledged, OOS of this audit but listed for future considerations.
---------------	---

V3ProviderOracle

[M07] Stale fee accounting in V3Provider pricing causes underestimated collateral value across health checks, liquidations, and slisBNBx rebalancing

Contract V3ProviderOracle.sol

Severity Medium

Description

The adapter's positionAmountsAt view values the managed position from the NonfungiblePositionManager's cached tokensOwed0 and tokensOwed1 plus the principal liquidity and idle inventory. Swap fees that have accrued in the pool since the last poke sit in the pool's fee growth accumulators and are not reflected in the cached owed values, and positionAmountsAt does not simulate them. As a result, every consumer of this view reports a value that is biased low by the amount of fees earned since the last poke.

JavaScript

```
/// @inheritdoc IOracle
function peek(address token) external view returns (uint256) {
    if (token != PROVIDER_SHARE) {
        return IOracle(resilientOracle).peek(token);
    }

    uint256 supply = IERC20(PROVIDER_SHARE).totalSupply();
    if (supply == 0) return 0; // pre-market

    // Fair composition from the adapter, taken at its
    // manipulation-resistant fair price
    // (exchange-rate-implied for slisBNB; pool TWAP clamped to the rate
    // for wstETH/wbETH; never raw
    // pool spot/slot0).
    (uint256 total0, uint256 total1) =
    IV3DexAdapter(ADAPTER).positionAmountsAt(
        IV3DexAdapter(ADAPTER).fairSqrtPriceX96()
```

Description

Impact

As such, the oracle peek used by Moolah for collateral pricing can underestimate the share price (depending on how inactive the vault is), and getUserBalanceInBnb used by the slisBNBx minter understates the user's balance. The bias is always downward, so it never lets a borrower over borrow and it never threatens protocol solvency, but it does impact several flows. Additionally, deposit compounds fees before pricing, so it is correct.

1. The withdrawal entry points price on stale value before fee compounding.

- a. In withdraw, the fee compounding runs after the Moolah health check and after the reward sync.
 - b. In withdrawShares, it never compounds at all while still triggering a health check. Because the collateral is valued slightly low, a position that is genuinely healthy can be judged unhealthy, so a legitimate withdrawal can revert, and a borrower near the loan to value limit can be liquidated earlier than fair. Additionally, for a successful withdrawal, within withdrawShares, an overestimated value of slisBNBX may be rebalanced (burned) from the user due to getUserBalanceInBnb pricing the user balance in BNB as lower than expected.
2. In the liquidation path, Moolah prices the seized collateral through peek, which is a view accessed by staticcall and therefore cannot perform the provider's fee compounding state changes after the seizure has already been priced. The liquidator seizes collateral at the underestimated share price, and when they exit through redeemShares the fees are compounded and realized.

Consider the following:

Recommendation

- In withdraw, perform the collectAndCompound call before performing collateral withdrawal
- In withdrawShares, add the collectAndCompound call before performing collateral withdrawal
- For the liquidation path, add a BOT only collectAndCompound entry point on the provider that forwards to the adapter's compound function under the existing BOT role, as suggested in [\[M01\]](#). The BOT can role can monitor swap volumes and perform routine compounding to ensure liquidation is performed at the most updated price.

Status

Fixed, now introduce a _pendingFees function that incorporates uncompounded pending fees on the V3 pool side within positionAmountsAt and used in peek to compute the total conservative pending fees to value the V3DexAdapter held NPM position accurately.

[L11] Unguarded zero rounding of the share price in peek() can enable free liquidation seizure or block liquidation

Contract V3ProviderOracle.sol

Severity **Low**

Description

Description

The V3ProviderOracle.peek() function computes the collateral share price as $raw = totalValue * 10^{** SHARE_DECIMALS} / supply$ and returns it after applying the haircut. The function reverts when either leg price is 0 and when $totalValue == 0$, but it has no equivalent guard for the case where raw itself floors to 0. When raw rounds down to 0, peek() silently returns 0 instead of reverting.

JavaScript

```
uint256 price0 = IOracle(resilientOracle).peek(TOKEN0); // 8 decimals
(rate-derived for the LST leg)
uint256 price1 = IOracle(resilientOracle).peek(TOKEN1); // 8
decimals
if (price0 == 0 || price1 == 0) revert ZeroPrice(); // finding D

uint256 totalValue = (total0 * price0) / (10 ** DECIMALS0) + (total1
* price1) / (10 ** DECIMALS1);
if (totalValue == 0) revert ZeroPrice(); // finding D

// 8-decimal USD price per ONE WHOLE share (10 ** SHARE_DECIMALS
share-wei) - Moolah interprets peek()
// using collateralToken.decimals() - minus the conservative
haircut.
uint256 raw = (totalValue * (10 ** SHARE_DECIMALS)) / supply;
return (raw * (BPS - haircutBps)) / BPS;
```

For a high value accounting asset such as slisBNB or WBNB, this requires the position to lose almost all of its value and is effectively unreachable for a correlated pair, so the current slisBNB/WBNB deployment is not exposed in practice. For a low USD value accounting asset the condition becomes easy to reach on a modest value drop, and an accounting asset worth $\leq 1e-8$ USD makes raw floor to 0 on any value decrease while the position still holds real value. This makes the issue a latent high severity risk for any reuse of the generic oracle with a cheap accounting asset, and it compounds the earlier concern about the unconstrained accounting asset.

Impact

When peek() returns 0, it becomes the collateral base price in Moolah, so the collateral price resolves to 0 and several market actions break. Borrowing is blocked because maxBorrow computes to 0, so no position can open or increase debt. Collateral withdrawal is blocked for any borrower with outstanding debt, since the health check treats the position as maximally unhealthy, locking the collateral. Liquidation is the most dangerous case. If the liquidator specifies seizedAssets > 0, the quoted value and the resulting repaidShares both compute to 0, so the liquidator seizes collateral while repaying zero debt, and if this drives collateral to 0 the borrower's remaining debt is written off as bad debt socialized to suppliers. If the position still held real value, this is direct theft of the borrower's collateral plus supplier losses. If instead the liquidator specifies repaidShares > 0, the seized amount calculation divides by the zero collateral price and reverts, bricking liquidation of that position and freezing the resulting bad debt.

Recommendation

Add a guard that reverts when raw == 0, mirroring the existing zero price and zero total value checks, so Moolah can never price the collateral at exactly 0 and the oracle degrades gracefully to an unavailable state instead of enabling free seizure or a division by zero.

Fixed, introduces a new price rounding feature that prevents peek from reverting that prevents liquidations, by enforcing a minimum price of 1.

Status

- It only fires on genuine dust. $\text{price} == 0$ is equivalent to true value $< 1e-8$ USD per whole share. Allowed downstream as an ordinary tiny price.
- It removes a real div-by-zero. $\text{peek} \geq 1$ keeps the derived collateralPrice nonzero, so the liquidation repaidShares path (which divides by collateralPrice) can't revert the way $\text{peek} == 0$ would.

[L12] Independent per-leg rounding in peek() slightly underprices collateral value

Contract V3ProviderOracle.sol

Severity Low

Description

The V3ProviderOracle.peek() function computes the position's total value as $(\text{total0} * \text{price0}) / (10^{**} \text{DECIMALS0}) + (\text{total1} * \text{price1}) / (10^{**} \text{DECIMALS1})$. Each leg is divided and floored on its own before the two are added, so every leg loses up to < 1 unit of 8-decimal USD. As a result totalValue can be underpriced by at most $-2e-8$ USD per call.

JavaScript

```
uint256 totalValue = (total0 * price0) / (10 ** DECIMALS0) + (total1 * price1) / (10 ** DECIMALS1);  
if (totalValue == 0) revert ZeroPrice(); // finding D
```

Description

Impact

The bound also holds for a heavily asymmetric, single-sided position, meaning a small total0 paired with a large total1 or the reverse. This is a normal operating state, since a rate-implied position sitting near the edge of its range becomes almost entirely one token, and it is the common case in which the smaller leg's floored contribution drops to 0 completely. Even then the per-leg loss is unchanged. The floor discards only the fractional part below 1 unit of 8-decimal USD, so the amount lost on any leg is always < 1 unit of 8-decimal USD regardless of how large or small that leg is. The total across both legs therefore stays bounded by $-2e-8$ USD, and relative to the dominant leg the zeroed dust leg is negligible.

The rounding direction is conservative. Rounding down undervalues the collateral, so a borrower's maximum borrow and health ratio come out marginally lower, making the position slightly more liquidatable. The rounding can never over-value collateral, so it cannot create bad debt.

Recommendation	This is optional hardening, since the current round-down favors the protocol and carries no security risk. If tighter precision is desired, defer both divisions to the end using full-precision math, for example computing each leg as <code>FullMath.mulDiv(total_i * price_i, 10 ** SHARE_DECIMALS, supply * 10 ** DECIMALS_i)</code> . This shrinks the absolute error to under 1 wei of the per-share price.
Status	Acknowledged, no fix implemented. This is acceptable considering it is unlikely that the currently supported LST pair markets will hold such a single-sided position, combined with the fact that ListaDao will be the initial depositor and the valuation loss is limited to a very small amount.

[I29] Instant haircut changes in `setHaircutBps()` can reprice all positions immediately

Contract	V3ProviderOracle.sol
Severity	Informational
Description	Description The <code>V3ProviderOracle.setHaircutBps()</code> function allows the <code>MANAGER</code> role (3/6 multisig) to change <code>haircutBps</code> to any value from 0 to 1000 basis points (0% to 10%) immediately.. Because <code>peek()</code> applies this haircut as a linear discount to the share collateral price, and Moolah derives every position's health and liquidation math from that price, a single call instantly reprices all live positions in the market. <pre> JavaScript /// @inheritdoc IV3ProviderOracle function setHaircutBps(uint256 _haircutBps) external onlyRole(MANAGER) { if (_haircutBps > MAX_HAIRCUT_BPS) revert InvalidHaircut(); haircutBps = _haircutBps; emit HaircutChanged(_haircutBps); } </pre> Impact Raising the haircut (for example from 0% to 10%) immediately reduces every collateral valuation by up to 10%, which can push positions that were healthy into a liquidatable state. Those borrowers are then liquidated while paying both the market's liquidation incentive and the haircut, since seized shares redeem at full undiscounted value and the liquidator captures the difference. This also exposes the affected positions to MEV driven forced liquidations. Lowering the haircut has the opposite effect: it inflates collateral valuations, enabling fresh over borrowing and eroding the safety buffer that protects the protocol against LP redemption slippage and valuation error.
Recommendation	Consider only allowing the <code>DEFAULT_ADMIN_ROLE</code> (Timelock) to call the <code>setHaircutBps</code> instead. Any emergency depeg of LSTs should be handled with other external pauses instead of the reliance on the <code>setHaircutBps</code> function.

Status	Acknowledged, no fix implemented. It is presumed that the MANAGER role would monitor V3 Collateral market pairs and their associated pool price appropriately before adjusting the haircutBPS parameter.
---------------	--

[I30] initialize does not emit HaircutChanged when setting the initial haircut

Contract	V3ProviderOracle.sol
Severity	Informational
Description	Description The V3ProviderOracle.initialize function sets the haircut value during contract initialization but does not emit the HaircutChanged event when it does so. <pre> JavaScript function initialize(address _admin, address _manager, address _resilientOracle, uint256 _haircutBps) external initializer { if (_admin == address(0) _manager == address(0) _resilientOracle == address(0)) revert ZeroAddress(); if (_haircutBps > MAX_HAIRCUT_BPS) revert InvalidHaircut(); __AccessControl_init(); _grantRole(DEFAULT_ADMIN_ROLE, _admin); _grantRole(MANAGER, _manager); resilientOracle = _resilientOracle; haircutBps = _haircutBps; } </pre> Impact Later updates to the haircut emit this event, so off chain systems, indexers, and monitoring tools that track haircut changes through event logs will miss the initial value. This leaves a gap in the event history, since the starting haircut is never recorded on chain as an event, making it harder to reconstruct the full sequence of haircut values from logs alone.
Recommendation	Consider emitting the HaircutChanged event inside initialize at the point where the initial haircut is set, mirroring the event emission already performed in the regular haircut update function.
Status	Fixed, now emits the HaircutChanged event within the initialize function.

[I31] Reuse of ZeroPrice() misrepresents the cause of the totalValue revert

Contract	V3ProviderOracle.sol
Severity	Informational
Description	Description In V3ProviderOracle.peek(), the guard if (totalValue == 0) revert ZeroPrice(); reuses the ZeroPrice error, but by the time execution reaches this line both price0 and price1 have already been confirmed non-zero by the earlier guard if (price0 == 0 price1 == 0) revert ZeroPrice();. Since the prices are already known to be non-zero here, totalValue == 0 can never indicate that a price feed returned zero. It can only mean one of two position-state conditions: the fair composition is empty, meaning total0 == 0 && total1 == 0, or both legs are non-zero but round down to 0 after the per-leg floor division. <pre> JavaScript uint256 totalValue = (total0 * price0) / (10 ** DECIMALS0) + (total1 * price1) / (10 ** DECIMALS1); if (totalValue == 0) revert ZeroPrice(); // finding D </pre> Impact Both of these are zero-value or degenerate-position conditions, not zero-price conditions. Reusing the ZeroPrice error therefore conflates a feed integrity failure, which the earlier guard handles, with a position state failure, which this guard handles. The result is semantically misleading and hampers incident response, because a reader cannot tell from the revert reason whether the oracle feed is broken, which points to investigating the data source, or the vault position is empty or degenerate, which points to investigating the vault. This is a naming and semantic clarity issue rather than a logic bug, since the revert itself is the correct behaviour.
Recommendation	Consider giving this revert its own error that names the actual cause, for example renaming it to ZeroValue().
Status	Fixed, removes the totalValue == 0 checks in place of new price rounding feature that prevents peek from reverting that prevents liquidations. The new price = 1 floor is safe as - It only fires on genuine dust. price == 0 is equivalent to true value < 1e-8 USD per whole share. Allowed downstream as an ordinary tiny price. - It removes a real div-by-zero. peek ≥ 1 keeps the derived collateralPrice nonzero, so the liquidation repaidShares path (which divides by collateralPrice) can't revert the way peek == 0 would.

Libraries (V3PositionLib & SwapInventoryLib)

[M01] Unconditional token approvals in mint and increaseLiquidity can block single sided operations

Contract V3PositionLib.sol

Severity Medium

Description

In V3PositionLib, the mint and increaseLiquidity functions call safeIncreaseAllowance for both token0 and token1 without first checking whether amount0Desired and amount1Desired are non zero. As a result, an approval is always issued for both tokens even when only one side is being supplied. This affects the flows in V3DexAdapter and V3Provider that reach these functions, including a single sided deposit and a single sided rebalance triggered by the BOT, where one of the two desired amounts is zero.

Description

JavaScript

```
function mint(
    INonfungiblePositionManager npm,
    address token0,
    address token1,
    uint24 fee,
    int24 tickLower,
    int24 tickUpper,
    uint256 amount0Desired,
    uint256 amount1Desired,
    uint256 amount0Min,
    uint256 amount1Min
) external returns (uint256 tokenId, uint128 liquidity, uint256
amount0, uint256 amount1) {
    IERC20(token0).safeIncreaseAllowance(address(npm), amount0Desired);
    IERC20(token1).safeIncreaseAllowance(address(npm), amount1Desired);

function increaseLiquidity(
    INonfungiblePositionManager npm,
    address token0,
    address token1,
    uint256 tokenId,
    uint256 amount0Desired,
    uint256 amount1Desired,
    uint256 amount0Min,
    uint256 amount1Min
) external returns (uint128 liquidity, uint256 amount0, uint256
amount1) {
    IERC20(token0).safeIncreaseAllowance(address(npm), amount0Desired);
    IERC20(token1).safeIncreaseAllowance(address(npm), amount1Desired);
```

Impact

For tokens that behave like [BNB on Mainnet](#), an approve(0) call fails. Because the code approves the zero side unconditionally instead of skipping it, this failure reverts the whole operation. The consequence is that a depositor can be permanently prevented from making a single sided deposit, and the same failure can block single sided withdrawals and single sided BOT rebalances.

JavaScript

```
/* Allow another contract to spend some tokens in your behalf */
function approve(address _spender, uint256 _value)
    returns (bool success) {
    if (_value <= 0) throw;
    allowance[msg.sender][_spender] = _value;
    return true;
}
```

Recommendation

Before calling safeIncreaseAllowance for each token, check that the corresponding desired amount is greater than 0, and only approve the side that is actually being supplied.

Status

Acknowledged, ListaDao has stated that only tokens that does not exhibit such behavior will be allowed for a V3 Collateral LP market

Note that this means that on mainnet, native a possible BNB token pair cannot perform single sided liquidity actions (rebalancing/liquidity addition/liquidity removals), so we likely do not want to list this type of markets.

[L13] Hardcoded block.timestamp deadline leaves user liquidity operations without time bound protection

Contract V3PositionLib.sol

Severity Low

Description

Description

V3PositionLib passes deadline = block.timestamp to the position manager for mint, increaseLiquidity, and decreaseLiquidity. The position manager enforces the deadline as require(block.timestamp <= deadline), so when the deadline equals block.timestamp the check is always satisfied in whatever block the transaction is finally mined. The deadline therefore provides no protection, and the operation can execute at any later time.

JavaScript

```
/// @dev Approve `npm` for both tokens and add liquidity to an existing position.
function increaseLiquidity(
    INonfungiblePositionManager npm,
```

```

        address token0,
        address token1,
        uint256 tokenId,
        uint256 amount0Desired,
        uint256 amount1Desired,
        uint256 amount0Min,
        uint256 amount1Min
    ) external returns (uint128 liquidity, uint256 amount0, uint256
amount1) {
        IERC20(token0).safeIncreaseAllowance(address(npm), amount0Desired);
        IERC20(token1).safeIncreaseAllowance(address(npm), amount1Desired);
        return
            npm.increaseLiquidity(
                INonfungiblePositionManager.IncreaseLiquidityParams({
                    tokenId: tokenId,
                    amount0Desired: amount0Desired,
                    amount1Desired: amount1Desired,
                    amount0Min: amount0Min,
                    amount1Min: amount1Min,
                    deadline: block.timestamp
                })
            );
    }

    /// @dev Remove `liquidity` from the position (tokens are accounted to
tokensOwed; collect separately).
    function decreaseLiquidity(
        INonfungiblePositionManager npm,
        uint256 tokenId,
        uint128 liquidity,
        uint256 amount0Min,
        uint256 amount1Min
    ) external {
        npm.decreaseLiquidity(
            INonfungiblePositionManager.DecreaseLiquidityParams({
                tokenId: tokenId,
                liquidity: liquidity,
                amount0Min: amount0Min,
                amount1Min: amount1Min,
                deadline: block.timestamp
            })
        );
    }
}

```

The user facing entry points `deposit`, `withdraw`, and `redeemShares` do not accept a `deadline` parameter of their own either, so a user cannot bound how long their transaction remains valid. As a result, the only remaining protection for these operations is the `amount0Min`, `amount1Min`, `minAmount0`, and `minAmount1` slippage bounds, which the caller computes against the price at submission time and which become stale while the transaction is pending.

Additionally, the adapter manages a single Uniswap V3 / PancakeSwap V3 position. When the BOT calls `rebalance()`, it collects and burns the old position, optionally runs a backend built inventory conversion swap, and mints a new position centered on the exchange rate within a range of $\pm 1\%$ (`INITIAL_RANGE_BPS = 100`). This recenters the composition of the managed position. A withdrawal/deposit that a user submitted earlier, with `minAmount0` and `minAmount1` computed from the composition before the rebalance, can therefore execute against the new position after the rebalance lands.

The $\pm 1\%$ figure bounds the range width, not the per token composition delta a pending withdrawal sees, since the rebalance can recenter from an off center prior range and run a swap bounded only by the BOT's own `amountOutMin`. However, the caller's `minAmount0` and `minAmount1` are forwarded into `decreaseLiquidity()`, which reverts if the post rebalance amounts fall short, and the share to liquidity math stays pro rata across a rebalance, so no dilution occurs

Impact

When a transaction is delayed, for example due to low gas, mempool congestion, or a validator deliberately withholding it, several consequences follow. A block producer or searcher can hold the transaction and release it at the least favorable moment that still satisfies the stale slippage bounds, extracting the difference. If the slippage bounds were set loosely, the operation fills at an outdated price. If the price has moved beyond the stale bounds, the transaction reverts only after the long delay, wasting the user's gas and silently failing their intent.

The impact is bounded because the slippage minimums are enforced, and because shares are minted and redeemed at the rate implied fair price rather than the pool spot, so a stale spot price does not misvalue shares. The rebalance path is not affected, since it enforces its own caller supplied deadline at the adapter level. For generic pairs whose share value is not anchored to an exchange rate, or that use loose default slippage bounds, the stale execution risk is larger.

Consider adding a deadline parameter to `deposit`, `withdraw`, `redeemShares`, and the other user facing collateral functions, and thread it through `V3PositionLib` to the position manager instead of hardcoding `block.timestamp`.

Recommendation

However, we should ensure that this deadline does not impact the NPM position `collectAndCompound` that can potentially allow a stale slippage to be used to compound fees from pending transactions.

Status

Not Fixed, no deadline parameter is added to the above `deposit`, `withdraw` and `redeemShares`, possibly allowing stale slippages when user perform actions related to V3 Pool actions (`addLiquidity/removeLiquidity`).

[I32] Incorrect V3PositionLib documentation misidentifies address(this) as the provider

Contract V3PositionLib.sol

Severity Informational

Description

The V3PositionLib header comment states that inside every function `address(this)` is the provider, and that token custody, allowances, and the collect recipient all resolve to the provider. This does not match the audited architecture. The adapter, not the provider, delegates calls to this library and is the sole NFT custodian, so `address(this)` actually resolves to the adapter. Token custody, allowances, and the collect recipient therefore all resolve to the adapter.

Description

JavaScript

```
* @dev The library is invoked via DELEGATECALL, so inside every
function `address(this)` is the
* provider: token custody, allowances and the collect
`recipient` all resolve to the
* provider, exactly as the inline code did. The provider's
immutables (NPM address, token
* addresses, fee) are not readable from library code, so they
are passed in as arguments.
* `recipient` is always `address(this)` and `deadline` always
`block.timestamp`, matching
* the previous inline behaviour.
*/
```

Impact

The problem is purely in the documentation. An auditor or maintainer reasoning from the comment would build a wrong mental model of where the NFT, the token allowances, and the collected fees actually land.

Recommendation

Update the V3PositionLib header comment so it states that `address(this)` is the adapter when the library is delegatecalled, and that token custody, allowances, and the collect recipient all resolve to the adapter as the sole NFT custodian.

Status

Not Fixed, the `address(this)` is still indicating V3PositionLib.sol function actions for the V3Provider contract, not the V3DexAdapter contract.

[I33] Residual NPM allowance grows in V3PositionLib

Contract V3PositionLib.sol

Severity Informational

Description

Description

In V3PositionLib, the `mint` and `increaseLiquidity` functions grant the `NonfungiblePositionManager` an allowance using `safeIncreaseAllowance(npm,`

amountDesired). The position manager only consumes the used amount, while the residual of desired use is never cleared. As a result, the standing allowance to the NPM keeps growing across deposits and compounds rather than returning to zero after each operation.

JavaScript

```
* @dev    The library is invoked via DELEGATECALL, so inside every
function `address(this)` is the
*        provider: token custody, allowances and the collect
`recipient` all resolve to the
*        provider, exactly as the inline code did. The provider's
immutables (NPM address, token
*        addresses, fee) are not readable from library code, so they
are passed in as arguments.
*        `recipient` is always `address(this)` and `deadline` always
`block.timestamp`, matching
*        the previous inline behaviour.
*/
```

Impact

The NPM is a trusted contract and the adapter pre-funds the exact amounts it works with, so the leftover allowance does not expose funds. This approach diverges from the pattern used in SwapInventoryLib, which applies forceApprove() followed by forceApprove(0) to leave no residual allowance. Carrying an ever increasing allowance is poor practice and makes the contract's approval state harder to reason about.

Recommendation

Consider resetting the allowance to 0 after the operation completes, or switch to the forceApprove pattern already used in SwapInventoryLib. This keeps the NPM allowance from accumulating and aligns the approval handling across the codebase.

Status

Acknowledged, no fix implemented, ListaDao has indicated that residual allowance to the trusted NPM contract is acceptable.

[I34] Native-handling in SwapInventoryLib.swap assumes a wrapped-native leg causing a possible DoS for non-native pairs

Contract SwapInventoryLib.sol

Severity Informational

Description

Description

SwapInventoryLib.swap re-wraps any native coin delivered during a swap back into the wrapped-native ERC-20, and reverts with UnexpectedNative when native arrives but neither tokenIn nor tokenOut is the wrapped-native token. This reflects the design assumption that exactly one leg of the pair is the wrapped native. The revert propagates through _convertToOptimalRatio into rebalance, so if native can be made to

arrive during the swap on a pair where neither token is the wrapped native, every rebalance reverts, which is a repeatable denial of service.

JavaScript

```
// Wrap any native this call delivered - a native-out venue's
proceeds (instantWithdraw → BNB) or a
// native-in venue's unspent refund - back into the wrapped-native
ERC-20, so it is booked into the
// totals via the spent/received deltas and never stranded. Native
can only belong to the
// wrapped-native leg; if neither leg is it, there is nowhere to
book the native ⇒ revert.
if (address(this).balance > beforeNative) {
    if (tokenIn != wrappedNative && tokenOut != wrappedNative) revert
    UnexpectedNative();
    IWBNB(wrappedNative).deposit{ value: address(this).balance -
    beforeNative }();
}
```

For the currently deployed adapters this branch is unreachable. SlisBNBV3DexAdapter, WstETHV3DexAdapter, and WbETHV3DexAdapter all pin TOKEN1 to the wrapped-native token (WBNB, WETH, and WETH respectively).

Impact

The issue is latent for a future non-native pair deployed on the abstract base V3DexAdapter. The base constructor requires WRAPPED_NATIVE != address(0) but does not require either token to equal it, so a pair where neither leg is the wrapped native can be deployed. For such a pair, any native delivered during the swap triggers the revert. Native can only reach the adapter through the wrapped-native unwrap or through a whitelisted swap pair, because the receive() function rejects all other senders, and a donation made before the swap is captured in beforeNative and does not trip the delta check, so the native must arrive during the swap itself.

A whitelisted router that forwards its own (attacker-donated) native balance to the caller, or that refunds native dust, therefore breaks rebalancing for that non-native pair, and an attacker can weaponize it by donating native to the router (For example: [Aerodrome SlipStreamRouter](#)) In that case the DoS would be Medium severity, while for the current scope it is Informational.

A related edge exists even for the in-scope pairs. When the swap sells the wrapped-native leg (tokenIn == wrappedNative), the re-wrap increases tokenIn's balance, and spent = beforeIn - afterIn can underflow and revert if the re-wrapped native delivery exceeds the amount the swap actually spent. This requires a whitelisted venue to deliver native during a sell-wrapped-native swap, which is atypical, but it is a theoretical donation-triggered revert in that direction.

Recommendation	Consider not whitelisting routers with a characteristic that auto refunds native ETH/BNB tokens allowing such griefing to occur.
Status	Fixed, now only redeposits native ETH/BNB for instantWithdrawals/refunds from swaps if either the input or output token is a wrapped native pair

LST Adapters (SlisBNB, WstETH & WbETH)

[M08] _lstNativeRate prices wstETH using stEthPerToken without accounting for a stETH/ETH depeg

Contract WstETHV3DexAdapter.sol

Severity Medium

Description

The WstEthV3DexAdapter._lstNativeRate function values wstETH by treating stEthPerToken (the stETH per wstETH rate) as if it were the WETH per wstETH rate. This relies on the assumption that stETH is worth ETH at a 1:1 ratio. The code acknowledges this assumption and documents that the stETH/ETH depeg is intentionally not priced at this layer, with the risk instead being absorbed by a lower LLTV.

Description

JavaScript

```
/// @dev WETH-per-wstETH (1e18) from Lido's on-chain accounting
(stETH≈ETH 1:1). Monotonic, not
/// market-driven - manipulation-resistant.
function _lstNativeRate() internal view override returns (uint256) {
    return IWstETH(WSTETH).stEthPerToken();
}
```

Impact

While this is a deliberate design choice, it means that whenever stETH trades below ETH, the fair price reported by the adapter overvalues the wstETH leg by the full magnitude of the depeg. A historical reference point is the -6% gap seen in June 2022. If the depeg exceeds the safety margin built into the LLTV, positions collateralized by wstETH can become undercollateralized while still appearing healthy, exposing the protocol to bad debt. The design is only safe if the LLTV buffer genuinely covers a realistic depeg scenario.

Confirm that the LLTV for wstETH collateral was sized against a realistic worst case stETH/ETH depeg, using historical events such as the June 2022 -6% gap (and ideally a more conservative stress figure) as a baseline.

Recommendation

Document the assumed maximum depeg and the resulting buffer so the parameter can be reviewed as market conditions change. If the buffer cannot be shown to cover a realistic depeg, incorporate the stETH/ETH market rate directly into the pricing path so the wstETH leg reflects the actual value of stETH rather than assuming a fixed 1:1 peg.

Status

Acknowledged, it is presumed that appropriate conservative LLTV will be set for the V3 LP Collateral LST markets. ListaDao has also indicated that depeg monitoring is in place to prevent such issues from occurring.

[M09] Unguarded pool.observe() in _twapTick() bricks collateral pricing for young or low-cardinality V3 pools

Contract WstETHV3DexAdapter.sol, WbETHV3DexAdapter & V3DexAdapter

Severity Medium

Description

The fairSqrtPriceX96() valuation reads the pool TWAP through _twapTick(), which calls pool.observe([TWAP_PERIOD, 0]) with no error handling. The lista-v3 pool oracle reverts with 'OLD' whenever the oldest retained observation is newer than (now - TWAP_PERIOD).

JavaScript

```
function _twapTick() internal view returns (int24 twapTick) {
    uint32[] memory secondsAgos = new uint32[](2);
    secondsAgos[0] = TWAP_PERIOD;
    secondsAgos[1] = 0;
    (int56[] memory tickCumulatives, ) =
    IListaV3Pool(P00L).observe(secondsAgos);
    int56 delta = tickCumulatives[1] - tickCumulatives[0];
    twapTick = int24(delta / int56(uint56(TWAP_PERIOD)));
    if (delta < 0 && (delta % int56(uint56(TWAP_PERIOD)) != 0))
    twapTick--;
}
```

Description

JavaScript

```
// ensure that the target is chronologically at or after the oldest
observation
require(lte(time, beforeOrAt.blockTimestamp, target), 'OLD');
```

A newly created pool is initialized with observationCardinality = 1 and a single observation stamped at the creation block. As a result, for the entire first TWAP_PERIOD of the pool's life (1800s = 30 minutes in the tested configuration), observe() reverts no matter what the cardinality is, because no observation older than (now - TWAP_PERIOD) can exist yet.

Calling increaseObservationCardinalityNext() does not help during this window, since cardinality only controls how many observations are retained once enough time has passed, not how far back history reaches. Even after the pool becomes older than TWAP_PERIOD, observe() still reverts when the cardinality is too small and the pool is actively traded, because with cardinality = 1 the single observation slot is overwritten with a recent observation on every new block.

JavaScript

```
/// @inheritdoc IListaV3PoolActions
function increaseObservationCardinalityNext(uint16
observationCardinalityNext)
    external
    override
    lock
    noDelegateCall
{
    uint16 observationCardinalityNextOld =
slot0.observationCardinalityNext; // for the event
    uint16 observationCardinalityNextNew =
        observations.grow(observationCardinalityNextOld,
observationCardinalityNext);
    slot0.observationCardinalityNext =
observationCardinalityNextNew;
    if (observationCardinalityNextOld !=
observationCardinalityNextNew)
        emit
IncreaseObservationCardinalityNext(observationCardinalityNextOld,
observationCardinalityNextNew);
}
```

1. First, the base `V3DexAdapter.fairSqrtPriceX96()` takes the pure-TWAP branch when `rate == 0` and returns a price derived directly from `_twapTick()`.
2. Second, the `WstETHV3DexAdapter` and `WbETHV3DexAdapter` overrides set `maxTwapDeviationBps` to `INITIAL_RANGE_BPS = 100` by default, which is non-zero, so the `dev == 0` pure-rate fast path is skipped and `_twapTick()` is always called to clamp the TWAP into `[rate*(1-dev), rate*(1+dev)]`.

Impact

The revert propagates from `fairSqrtPriceX96()` through

- `V3Provider.deposit`, since the `fairSqrtPriceX96` is utilized to determine valuation of liquidity added to perform share minting, and as such, users cannot perform collateral supply to perform borrowing actions
- `V3ProviderOracle.peek()`, which calls `positionAmountsAt(fairSqrtPriceX96())`, and therefore into every Moolah collateral price read. Moolah calls `peek()` on every borrow and collateral withdrawals health check so a `wstETH/WETH` or `wbETH/WETH` market, or any pure-TWAP pair, that is wired to a freshly created or low-cardinality `IListaV3` pool is bricked. Users cannot supply collateral and hence cannot borrow/withdraw collateral

	Note that the slisBNB pair is not affected, because its rate is always non-zero and its fair price is pure rate-implied and never calls observe().
Recommendation	Ensure that the V3Provider and Adapter contracts for newly deployed pools are only deployed after the V3 Pools are deployed with sufficient age e.g. at least older than TWAP_PERIOD seconds old.
Status	Acknowledged, ListaDao has indicated that appropriate operational deployment SOP requires increaseObservationCardinalityNext and TWAP health verification before launch, thus mitigating this issue.

[M10] slisBNB fair price prices no market depeg, leaving depeg risk entirely to LLTV unlike the wstETH and wbETH adapters

Contract	SlisBNBV3DexAdapter.sol
Severity	Medium
Description	Description SlisBNBV3DexAdapter does not override fairSqrtPriceX96, so it uses the base implementation, which for a nonzero rate returns <code>_sqrtPriceX96FromRate(rate)</code>, a pure rate-implied price with no pool read. <pre> JavaScript /// @inheritdoc IV3DexAdapter /// @dev Rate-implied (manipulation-resistant) when the subclass /// supplies a non-zero LST-native /// rate via _lstNativeRate(); otherwise falls back to the pool TWAP. function fairSqrtPriceX96() public view virtual returns (uint160) { uint256 rate = _lstNativeRate(); if (rate == 0) return TickMath.getSqrtRatioAtTick(_twapTick()); return _sqrtPriceX96FromRate(rate); } </pre> WstETHV3DexAdapter and WbETHV3DexAdapter instead override fairSqrtPriceX96 to take the pool TWAP and clamp it into $[\text{rate} * (\text{BPS} - \text{dev}) / \text{BPS}, \text{rate} * (\text{BPS} + \text{dev}) / \text{BPS}]$, where dev is maxTwapDeviationBps. The consequence is that slisBNB collateral is always valued at the accounting rate from StakeManager.convertSnBnbToBnb, with zero sensitivity to the market price, while wstETH and wbETH collateral tracks the market down to their clamp floor $\text{rate} * (\text{BPS} - \text{dev}) / \text{BPS}$ and so prices the first dev of any depeg. <pre> JavaScript function fairSqrtPriceX96() public view override returns (uint160) { </pre>

```

uint256 rate = _1stNativeRate();
uint256 dev = maxTwapDeviationBps;

// dev == 0 ⇒ pure rate-implied: skip the TWAP read entirely, so
the valuation has no pool
// observe()/observation-cardinality dependency.
if (dev == 0) return _sqrtPriceX96FromRate(rate);

uint160 sqrtLow = _sqrtPriceX96FromRate((rate * (BPS - dev)) / BPS);
uint160 sqrtHigh = _sqrtPriceX96FromRate((rate * (BPS + dev)) /
BPS);
uint160 twapSqrt = TickMath.getSqrtRatioAtTick(_twapTick());
if (twapSqrt < sqrtLow) return sqrtLow;
if (twapSqrt > sqrtHigh) return sqrtHigh;
return twapSqrt;
}

```

Additionally, the `convertSnBnbToBnb` is not enforced to be up-to-date within the `ListaStakeManager` given the exchange rate only advances when a permissioned BOT calls `compoundRewards()`, which further encourages a deviation check.

Impact

slisBNB is not immune to a market depeg.

- Instant redemption through `StakeManager.instantWithdraw` charges an instant-withdraw fee, so the arbitrage floor is $\text{rate} - \text{fee}$ rather than rate , which allows a persistent small discount below the rate.
- Instant redemption also depends on the `StakeManager` holding sufficient liquid BNB, and under mass redemption that reserve can deplete and redemptions fall back to delayed unbonding over several days, which can widen the discount to several percent, the same dynamic that produced the stETH depeg.
- A slashing event on the underlying stake could also reduce the rate itself.

Because the slisBNB fair price prices none of this, a slisBNB depeg is not reflected in the collateral valuation at all. During a depeg the oracle over-values slisBNB collateral at the rate, which lets borrowers over-borrow against depreciated collateral and creates bad debt on default, with the entire depeg carried by the market LLTV rather than partially absorbed by the price as it is for wstETH and wbETH.

Recommendation Consider the following

- Either confirm that the LLTV of the slisBNB market is sized to cover the maximum plausible slisBNB depeg, accounting for the instant-withdraw fee floor, a stress scenario where instant redemption is unavailable and unbonding is delayed, and a slashing haircut, and state that pure rate pricing is intentional because that buffer covers the depeg.
- If the slisBNB and WBNB pool has sufficient depth and observation cardinality for a trustworthy TWAP, adopt the same clamped-TWAP fairSqrtPriceX96 used by the wstETH and wbETH adapters, with a dev tuned to slisBNB's peg behavior, so that the first dev of any depeg is priced and the three adapters are consistent. The correct choice depends on the slisBNB and WBNB pool depth and the LLTV configured for that market, so verify both before deciding.

Acknowledged, no fix implemented. ListaDao has stated that monitoring systems are in place to detect depegs of slisBNB.

Status

It is presumed that sufficient market pause (can be performed via Moolah side whitelist or Market wide pause). If a V3Provider specific emergency pause mechanism is desired, consider inheriting OZ's PausableUpgradeable contract and assigning the PAUSER and unpauser (typically MANAGER) role appropriately.

[L14] TWAP clamp band in fairSqrtPriceX96() is not cross checked against the NPM position range,

Contract WstETHV3DexAdapter.sol & WbETHV3DexAdapter.sol

Severity Low

Description

In WstETHV3DexAdapter and WbETHV3DexAdapter (built on the base V3DexAdapter), fairSqrtPriceX96() prices the position by taking the pool TWAP and clamping it into $[\text{rate} \cdot (1 - \text{dev}), \text{rate} \cdot (1 + \text{dev})]$, where the anchor is the LIVE `_lastNativeRate()`. The actual liquidity, however, sits in a tick range `[tickLower, tickUpper]` that was centered on `lastCenterRate`, the rate captured at the previous rebalance.

Description

Nothing reconciles these two anchors. On top of that, `maxTwapDeviationBps` (the dev value, allowed up to `MAX_TWAP_DEVIATION_BPS = 1000 = 10%`) is configured independently of the range half width (`INITIAL_RANGE_BPS = 100 = 1%`) and of `centerRateThresholdBps`.

The clamp is only meaningful while its band overlaps the position's active range. The design relies on the invariant that the token split tracks real in range drift while the rate clamp bounds TWAP manipulation, and that invariant holds only during overlap. Two separate, unchecked conditions break it:

JavaScript

```
function fairSqrtPriceX96() public view override returns (uint160) {
    uint256 rate = _lastNativeRate();
    uint256 dev = maxTwapDeviationBps;

    // dev == 0 ⇒ pure rate-implied: skip the TWAP read entirely, so
    the valuation has no pool
    // observe()/observation-cardinality dependency.
    if (dev == 0) return _sqrtPriceX96FromRate(rate);

    uint160 sqrtLow = _sqrtPriceX96FromRate((rate * (BPS - dev)) / BPS);
    uint160 sqrtHigh = _sqrtPriceX96FromRate((rate * (BPS + dev)) /
BPS);
    uint160 twapSqrt = TickMath.getSqrtRatioAtTick(_twapTick());
    if (twapSqrt < sqrtLow) return sqrtLow;
    if (twapSqrt > sqrtHigh) return sqrtHigh;
    return twapSqrt;
}
```

Impact

The impact is bounded to Low for in-scope LST pairs because wstETH/WETH and wbETH/WETH are near parity pairs, with a rate of roughly 1.0 to 1.2, and the range is tight, so a full composition flip is a small move in USD terms, on the order of 0.005% in range over valuation. The mechanism is general and would be more severe on a pair whose two legs diverge in price if the base V3DexAdapter is inherited for more volatile pairs.

1. When dev is greater than the range half width. With dev = 10% and a range of +/-1%, the clamp band is 10x wider than the position. Any TWAP more than about 1% off center already lands outside [tickLower, tickUpper], so `getAmountsForLiquidity` returns a degenerate boundary composition of 100% of one token, while the 10% clamp does nothing to hold the mark inside the range. A TWAP manipulator can then pin the valuation to either boundary and flip the reported composition between all of token0 and all of token1.
2. Under discretionary rebalancing, `_requireCenterRateDeviation` is only a MINIMUM movement gate that permits a recenter once drift $\geq$ the threshold. Nothing ever forces a recenter, so the live rate, and the clamp band anchored to it, can drift arbitrarily far from `lastCenterRate` until the band no longer overlaps the range. At that point the clamp gives no protection relative to the actual liquidity.

Recommendation Consider the following

1. Bound maxTwapDeviationBps to the range half width, or verify at valuation time that the clamp band overlaps [tickLower, tickUpper], so the clamp stays engaged inside the range where the composition tracking assumption is valid.
2. Add an UPPER drift bound that caps how far the live rate may sit from lastCenterRate before the valuation is treated as stale or a rebalance is forced, complementing the existing lower anti churn gate rather than replacing it. Keep the clamp anchored to the live rate.

Status

Acknowledged, no fix implemented. It is presumed that off-chain checks will be performed related to current tick ranges of the associated dex adapter position and pair price before adjustments of maxTwapDeviationBps by the MANAGER role.

[I35] maxTwapDeviationBps is a live, un-timelocked oracle parameter that lets MANAGER swing V3Provider collateral valuation during a TWAP deviation

Contract

WstETHV3DexAdapter.sol

Severity

Informational

Description

In WstETHV3DexAdapter and WbETHV3DexAdapter, fairSqrtPriceX96() returns the pool TWAP clamped into $[\text{rate} * (\text{BPS} - \text{dev}) / \text{BPS}, \text{rate} * (\text{BPS} + \text{dev}) / \text{BPS}]$, where dev is maxTwapDeviationBps. This fair price feeds two consumers: V3Provider.deposit, which uses it to value the existing position and the newly added liquidity when minting shares, and V3ProviderOracle.peek, which uses it through positionAmountsAt to price the V3Provider share token as Moolah collateral.

maxTwapDeviationBps is set by setMaxTwapDeviationBps, which is restricted to the MANAGER role and bounded only by MAX_TWAP_DEVIATION_BPS = 1000 (10%). The change takes effect immediately, with no timelock and no smoothing.

Description

JavaScript

```

    /// @notice Set the TWAP-vs-rate clamp band (BPS) for the valuation
    price. 0 => pure rate-implied.
    function setMaxTwapDeviationBps(uint256 _maxTwapDeviationBps) external
    onlyRole(MANAGER) {
        if (_maxTwapDeviationBps > MAX_TWAP_DEVIATION_BPS) revert
        InvalidDeviation();
        maxTwapDeviationBps = _maxTwapDeviationBps;
        emit MaxTwapDeviationChanged(_maxTwapDeviationBps);
    }

```

Impact

	The oracle NAV is the position composition at the fair price valued at rate-derived leg prices, so shifting dev moves the collateral valuation by up to dev, which is at most 10%. This lever is active only when the TWAP has deviated from the rate, since when the TWAP sits inside the clamp band for both the old and new dev the returned price is unchanged. In other words, the manipulation window is exactly a depeg or TWAP deviation, which is when the lending system is already under stress. During such a deviation, a manager can raise dev so the clamp floor rate * (BPS - dev) / BPS drops, under-valuing collateral and triggering unfair liquidations of otherwise healthy borrowers, or lower dev so the price pins toward the higher rate, over-valuing collateral and allowing over-borrowing that becomes bad debt on default. Because the change is a single instantaneous transaction, a malicious or compromised MANAGER key can also sandwich a liquidation by setting dev, liquidating, then resetting dev.
Recommendation	Consider routing setMaxTwapDeviationBps through a timelock or governance so a valuation-affecting change cannot be applied atomically around a deposit or liquidation, and so market participants can observe pending changes
Status	Acknowledged, no fix implemented. It is presumed that the MANAGER role would monitor V3 Collateral market pairs and their associated pool price appropriately before adjusting the maxTwapDeviationBps parameter. It is also presumed that a safe LLTV will be implemented to account for possible fluctuations in V3 LP Collateral pricings before relying on a haircut for the oracle.

[I36] Dead-branch indirection in SlisBNBV3DexAdapter rate derivation can be compressed to a single call

Contract	SlisBNBV3DexAdapter.sol
Severity	Informational
Description	Description SlisBNBV3DexAdapter derives its exchange rate through two internal helpers, _isSlisBnbWbnbPool() and _poolPriceRate(), that _lstNativeRate() combines as _isSlisBnbWbnbPool() ? _poolPriceRate() : 0. Both helpers contain conditionals that are already resolved by the constructor, so they carry unreachable branches. Description The constructor pins the pair with if (!(_token0 == SLISBNB && _token1 == WBNB)) revert NotSlisBnbWbnbPair(), so after construction TOKEN0 is always SLISBNB and TOKEN1 is always WBNB, and there is no other deployable configuration. The inherited V3DexAdapter constructor also enforces _token0 < _token1 through the TokenOrderInvalid check, which is consistent with this pair because SLISBNB (0xB0b8...) is less than WBNB (0xbb4C...).

Under that fixed invariant, `_isSlisBnbWbnbPool()` always returns true, since its first condition (`TOKEN0 == SLISBNB && TOKEN1 == WBNB`) always holds and its second condition (`TOKEN0 == WBNB && TOKEN1 == SLISBNB`) is unreachable. Likewise `_poolPriceRate()` always returns `STAKE_MANAGER.convertSnBnbToBnb(1e18)`, since `TOKEN0 == SLISBNB` always holds and the `convertBnbToSnBnb(1e18)` branch is unreachable. The `: 0` fallback in `_lstNativeRate()` is therefore never taken.

JavaScript

```

    /// @dev slisBNB-WBNB rate from the StakeManager (1e18). 0 for any
    non-slisBNB/WBNB pair → base TWAP.
    function _lstNativeRate() internal view override returns (uint256) {
        return _isSlisBnbWbnbPool() ? _poolPriceRate() : 0;
    }

    /* ----- internals -----
    */

    function _isSlisBnbWbnbPool() internal view returns (bool) {
        return (TOKEN0 == SLISBNB && TOKEN1 == WBNB) || (TOKEN0 == WBNB &&
        TOKEN1 == SLISBNB);
    }

    function _poolPriceRate() internal view returns (uint256) {
        return TOKEN0 == SLISBNB ? STAKE_MANAGER.convertSnBnbToBnb(1e18) :
        STAKE_MANAGER.convertBnbToSnBnb(1e18);
    }

```

Impact

Both helpers are internal and are called only from `_lstNativeRate()`, so the whole path collapses to a single call. This also makes the adapter inconsistent with its siblings, since `WstETHV3DexAdapter` and `WbETHV3DexAdapter` return their rate directly through `stEthPerToken()` and `exchangeRate()` with no pool-direction or pool-detection scaffolding. There is no fund risk or behavioral difference, only unnecessary code, larger bytecode against the EIP-170 limit, and extra conditionals at runtime.

Recommendation Consider removing `_isSlisBnbWbnbPool()` and `_poolPriceRate()`, which matches the `wstETH` and `wbETH` adapters and removes the dead branches.

Status Fixed, the `_lstNativeRate` function is compressed to a singular `ListaStakeManager.convertSnBnbToBnb` call, with the assumption that exchange rates are never zero.

[I37] Unused IMoolah import in SlisBNBV3Provider

Contract SlisBNBV3DexAdapter.sol

Severity Informational

Description	Description The SlisBNBV3Provider contract imports IMoolah and Id from the same file, but IMoolah is never used anywhere in this contract. The Id type is used in mappings and function parameters, while all interactions with Moolah go through the MOOLAH variable inherited from the base V3Provider contract, whose type is already resolved through inheritance. The contract never casts to IMoolah or declares a local variable of that type, so the IMoolah symbol is redundant here.
	<pre>JavaScript import { IMoolah, Id } from "moolah/interfaces/IMoolah.sol";</pre>
	Impact This has no impact on functionality or security. It is a code cleanliness issue that adds clutter and can trigger an unused import warning during compilation.
Recommendation	Consider removing the IMoolah from the import statement while keeping Id, so the line becomes an import of only Id from the Moolah interfaces file.
Status	Not fixed, the unused IMoolah interface is not removed.

[I38] V3DexAdapter has no path to claim or recover future external LP incentives on its NPM position

Contract	V3DexAdapter(s)*
Severity	Informational
Description	Description Each V3DexAdapter is the sole custodian of a single concentrated liquidity NFT held through a canonical position manager, Uniswap V3 on Ethereum for the wstETH and wbETH adapters and PancakeSwap V3 on BSC for the slisBNB adapter. The only reward action the adapter performs is collectAndCompound, which collects the position's swap fees and folds them back into the position. As currently deployed, the specific pools these adapters use have no active external incentive program, so no external rewards are being forgone or stranded today, and this is not a live issue.
	Impact The gap is latent and forward looking. If any external incentive program is later launched for one of these pools, the adapter cannot capture it. Programs that airdrop a separate reward token to the position or NFT owner, such as a Merkl style concentrated liquidity campaign, would send that token to the adapter address, but the adapter tracks only idleToken0 and idleToken1 and exposes no function to withdraw or claim any other token, so those rewards would be stranded.

- Programs that require the NFT to be staked into a farm contract, such as a MasterChefV3 or a uniswap-v3-staker deployment, would earn nothing at all, because the adapter never stakes the NFT and would still need to keep it redeemable for withdrawals and liquidations. In both cases the outcome is forgone or unrecoverable yield rather than any loss of user funds or protocol solvency, which is why this is informational.

Note that wstETH and wbETH does not have a concept of a reward earning layer similar to SlisBNBx of the SlisBNBV3Provider.

Recommendation

If future reward participation is required using the underlying NPM position held by the WstETHV3DexAdapter and WbETHV3DexAdapter, an upgrade is required to be performed, and any relevant logic should be submitted for are-audit.

Status

Acknowledged, no fix implemented

LST Providers (SlisBNB, WstETH & WbETH)

No issues were discovered